

Amortized Deduction for Binary Linear Systems: Learning Branching Guidance from Exact Conditional Marginals

Binary linear system을 위한 amortized deduction:
정확한 conditional marginal로부터 branching guidance를 학습하기
neuroMCTS project · 2026년 9월 · 한국어판

초록

Binary linear system(BLS)은 $A \in \{0,1\}^{m \times n}$ 에 대해 $Ax=b$ 를 만족하는 $x \in \{0,1\}^n$ 를 찾는 문제이며, 레이저 기반 비파괴검사에서 정수 투영값으로부터 이진 점유 영상을 복원하는 과제로 나타난다. market-split 계열 인스턴스에서는 linear relaxation이 의도적으로 무정보하도록 설계되므로, backtracking search는 값싼 heuristic으로는 잘 내릴 수 없는 branching 결정에 대부분의 노력을 쓴다. 본 논문은 이 설정을 위한 학습 기반 branching heuristic과, 그것을 지도학습하기 위한 정확한 절차를 제시한다.

방법은 두 관찰에 기반한다. 첫째, BLS를 partial assignment로 conditioning하는 것은 그 배정을 대입해 더 작은 BLS를 얻는 것과 동일하다. 따라서 하나의 network가 구조 변경 없이 root와 모든 interior node에서 예측하며, 학습 상태는 크기가 줄어든 평범한 instance가 된다. 둘째, 전수열거가 가능한 크기의 instance에서는 정확한 conditional marginal $p_j = \Pr[x_j=1 \mid A, b]$ 를 계산할 수 있고, 이것이 node의 자유 변수를 두 부류로 분할한다: 살아남은 모든 해가 일치해 반대로 분기하면 subtree가 비어 backtrack이 확정되는 변수(DET)와, 해들이 갈려 어느 쪽으로 분기해도 해에 도달하는 변수(OPEN)다. 비용을 발생시키는 것은 전자뿐이다. 이 분할은 정확한 학습 target과 동시에 어떤 예측이 중요한지에 대한 기준을 제공한다: root에서는 변수의 6.3%만이 DET이지만 depth 8에서는 그 비중이 87.9%가 되며, 정확도가 탐색 비용으로 환산되기 시작하는 지점이 바로 여기다.

우리는 21.8K 파라미터의 bipartite graph network를 이 target에 학습시키고, 오직 branching에만 사용한다 — 증명이 아니라 확률을 내놓으므로 변수 확정에는 쓰지 않는다. 같은 변수를 탐지하는 건전 절차인 LP-probing과 비교하면, probing은 변수마다 linear program을 하나씩 푸는 반면 network는 relaxation 한 번과 forward pass 한 번이면 된다. 따라서 비용 비율이 instance 크기에 비례해 커진다: 양쪽 모두 warm start한 backend에서 측정해 $n=25$ 에서 1.7배, $n=60$ 에서 7.2배, $n=150$ 에서 17.8배다. wall-clock 예산을 맞추고 세 seed로 반복한 비교에서, warm start한 탐색 루프에서 21×60 기준 LP 기반 branching 대비 median 2.6~3.0배 빠르고(모든 seed에서 30개 중 23개에서 우세, sign test $p=0.005$), node를 9.5배 적게 전개하며, 두 전략 모두 모든 instance를 푼다. lattice reduction이 먼저 도는 전체 pipeline에서는 잔여 instance를 LP 기반 branching보다 3.0배 빠르게 닫고, random branching은 잔여를 하나도 닫지 못한다. solution 개수를 크기 간에 통제하면 transfer 손실이 측정되지 않는다: 10×25 에서만 학습한 network가 평가 크기에서 학습한 network와 통계적으로 구별되지 않는다. 21×60 100개 세트에서 지도학습 guidance는 LP 기반 branching보다 2.0~2.3배 빠르고 node를 5.5배 적게 쓴다. 추가 실험 둘이 지도 target이 무엇을 잡고 무엇을 놓치는지를 특정한다. network의 예측 이득이 사는 배포 크기의 상태로 학습하는 것은 도움이 되지 않는다: 대부분의 node가 소비되는 작은 interior 상태에서 능력을 잃어 탐색이 느려진다. 도움이 되는 것은 탐색이 실제로 지불하는 목적함수를 최적화하는 것이다. 모든 branching 결정의 정확한 subtree 비용을 탐색 자체에서 얻을 수 있으므로, 지도 규칙에서 출발한 branching policy의 actor-critic fine-tuning은 같은 100개 instance에서 지도 policy보다 node를 2.3배 적게 전개하고 2.0~2.2배 빠르며($p<0.01$), LP 기반 branching보다 4.8~5.9배 빠르다. fine-tuning된 policy는 held-out 상태의 95%에서 확신도 규칙과 다른 변수 — 대개 network가 덜 확신하는 변수 — 로 분기하는데, 이는 맞을 확률과 틀렸을 때의 비용이 서로 다른 목적함수라는 직접 증거다.

달성하지 못한 것도 함께 보고한다. CP-SAT는 동일 instance를 모든 크기에서 9~40배 빠르게(fine-tuning 후 21×60 에서 9~10배) 풀며, 우리 시스템은 그것과 경쟁하지 못한다. 그 격차를 분해하면 우리 branching이 node를 2.2배 적게 쓰고 node 처리는 32배 느리다 — 결함이 branching 품질이 아니라 node당 비용에 있다는 뜻이나, 그것이 회복 가능함을 보인 것은 아니다. 그리고 probing 대비 비용 우위는 점근적으로는 실재하나, 우리 instance에서 예측이 가장 중요한 depth에서는 1.3~1.7배에 그치며 그 구간에서 probing은 예측이 아니라 증명을 제공한다. 따라서 우리가 주장하는 기여는 방법론적이다: 정확한 conditional posterior는 branching에 대해 얻을 수 있고 잘 정의된 지도 신호이며, in-tree 상태가 root 상태보다 나은 학습 분포이며 — root로 학습한 network는 tree 안에 배치하면 relaxation rounding보다 못하고, in-tree로 학습한 network는 3.4%p 앞선다 — 그렇게 얻은 heuristic은 instance 크기를 넘어 전이된다. 코드, 생성기, 정확한 marginal label, 인스턴스 단위 결과를 공개한다.

1. 서론

1.1 문제

$A \in \{0,1\}^{m \times n}$, $b \in \mathbb{Z}^m$ 이 주어질 때 binary linear system(BLS)은 다음 feasibility 문제다:

$$Ax = b \text{ 를 만족하는 } x \in \{0,1\}^n \text{ 를 찾아라. (1)}$$

그 해집합을 $S = \{x \in \{0,1\}^n : Ax=b\}$ 라 쓴다. 각 row는 cardinality constraint로, 지정된 셀 부분집합 중 몇 개가 점유되어 있는지를 나타낸다.

응용 배경은 레이저 기반 비파괴검사다. 시편을 m 개 방향으로 스캔하면 각 스캔은 경로상 흡수 셀의 정수 개수를 반환하고, 과제는 이진 점유 영상을 복원하는 것이다. 이 설정의 두 성질이 문제 정식화를 규정한다. 해는 유일할 필요가 없다 — 동일한 측정치와 부합하는 영상이 여럿일 수 있고, S 의 아무 원소나 복원하면 물리적 문제는 해결된다. 그리고 측정 노이즈가 b 를 교란해 $S=\emptyset$ 이 될 수 있으므로, 신뢰할 수 있는 infeasibility 판정도 해를 찾는 것만큼 가치가 있다.

$S \neq \emptyset$ 판정은 일반적으로 NP-complete이다. 본 연구의 인스턴스는 market-split 문제를 본뜬 고난도 영역에서 의도적으로 추출되며, 그 영역에서는 linear relaxation $S_{LP} = \{x \in [0,1]^n : Ax=b\}$ 가 대부분 fractional한 vertex를 갖는 큰 polytope이다. 따라서 relaxation 기반 bounding이 약하고, complete solver는 relaxation의 도움을 거의 받지 못한 채 어디서 분기할지 결정해야 한다 — 이것이 본 논문이 학습하는 결정이다.

1.2 학습 컴포넌트가 붙는 자리, 그리고 무엇으로 학습할 것인가

식 (1)을 위한 complete solver는 deduction(constraint propagation, relaxation 기반 가지치기)과 branching 결정(어느 자유 변수로 분기하고 어느 값을 먼저 시도할 것인가)을 번갈아 수행한다. deduction은 건전해야 하고 앞으로도 그래야 하지만, branching은 heuristic한 선택이라 오류의 대가가 탐색 시간일 뿐 정확성이 아니다. 이 비대칭이 branching을 학습 컴포넌트가 붙기에 자연스러운 자리로 만들며, mixed-integer programming에서 학습이 가장 성공한 지점이기도 하다.

어려운 것은 지도(supervision)다. 표준적인 답이 두 가지 있는데 여기서는 둘 다 만족스럽지 않다. 심어진 해 x^* 로 학습하는 것은 S 의 한 원소를 ground truth로 취급하는데, $|S|>1$ 이면 이것은 잘못 정의된 문제다: (A,b) 의 어떤 것도 심어진 해를 다른 해와 구분하지 못하므로 label은 posterior가 아니라 posterior로부터의 표본이다. strong-branching imitation처럼 비싼 expert를 모방하는 것은 그 결정이 유익하면서도 수집 가능한 expert를 요구하는데, 여기서 자연스러운 후보인 LP-probing은 node당 변수당 linear program 한 번을 지불한다.

우리는 세 번째 길을 택한다. 이 인스턴스들이 planted solution을 갖고 적당한 크기에서 feasibility를 열거할 수 있기 때문에 가능한 길이다: 작은 인스턴스에서 전수열거로 정확한 conditional posterior를 계산하고 그것에 학습한다. §3은 이것이 실행 가능하며, 정확한 의미에서 올바른 target임을 보인다 — 그리고 그 posterior가 어떤 예측이 탐색에 영향을 줄 수 있는지까지 함께 식별함을 보인다.

1.3 기여

(1) in-tree 지도학습을 간단히 만드는 reduction 항등식. 식 (1)을 partial assignment로 conditioning하면 같은 구조의 더 작은 BLS가 된다(명제 1). 따라서 하나의 architecture, 하나의 label 생성기, 하나의 학습 루프가 root와 모든 interior node를 담당하며, depth- d 학습 상태는 $n-d$ 개 변수를 갖는 평범한 instance로 저장된다.

(2) 비용을 발생시킬 수 있는가에 따른 branching 결정의 분할. 정확한 marginal이 node의 자유 변수를 DET(모든 살아남은 해가 일치; 반대로 분기하면 subtree가 빔)와 OPEN(해들이 갈림; 어느 쪽이든 해에 도달)으로 나눈다. DET 오류만이 backtrack을 유발한다. 그 구성을 측정하면 root에서 6.3%, depth 8에서 87.9%다 (§4.3, §4.4). 이는 쓸모 있는 예측이 root가 아니라 tree 내부에 있음을 특정하며, root 수준 정확도가 왜 나쁜 목적함수인지 설명한다.

(3) 근사 대상인 건전 절차 대비 비용 특성 규명. network는 relaxation 한 번과 forward pass 한 번, LP-probing은 변수마다 relaxation 한 번이 필요하다. 양쪽 모두 warm start한 backend에서 측정하면 비율이 $n=25$ 의 1.7배에서 $n=150$ 의 17.8배로 커진다 (§4.5). 단일 수치가 아니라 스케일링 법칙으로 보고하며, 우리 탐색이 실제로 작동하는

크기에서는 우위가 크지 않고 probing에는 건전성이라는 추가 장점이 있다는 점도 함께 밝힌다.

(4) wall-clock 예산을 맞춘 종단 평가, 3-seed 반복. warm start한 탐색 루프에서 21×60 의 학습된 guidance는 LP 기반 branching 대비 100개 세트에서 median 2.0~2.3배 빠르고(70~71/100 우세, sign test $p < 0.001$) node를 5.5배 적게 전개한다. 원래의 30개 세트에서는 2.6~3.0배·9.5배이며, 두 전략 모두 600초 내 모든 instance를 푼다(§ 4.6). 실제 cascade에서는 두 건전 전략 모두 잔여 instance를 전부 닫되 학습된 쪽이 3.0배 빠르고(seed 합산 50개 중 40개 우세, $p = 2 \times 10^{-5}$), random branching은 잔여 50개 중 하나도 닫지 못한다(§ 4.12).

(5) solution multiplicity를 통제된 크기 transfer. $|S|$ 를 크기 간에 비슷하게 유지하면, 10×25 에서만 학습한 network가 평가 크기 21×60 에서 학습한 network보다 못하지 않다(30개 중 18~19개에서 더 빠르고 node도 18개에서 더 적으며, 어느 방향도 유의하지 않음). 학습된 양이 크기에 특수하지 않음을 보인다(§ 4.8).

(6) 학습 분포를 배포에 맞추는 것에 대한 부정 결과. network가 relaxation rounding을 앞서는 곳은 자유변수 50~60개 상태이며 학습 범위 13~25 밖이다. 다른 모든 것을 고정하고 정확히 그 상태로 학습하면 그곳의 정확도는 약 1%p 오르지만 search tree는 두 배가 된다 — 전체 node의 2/3가 소비되는 작은 interior 상태에서 능력을 잃기 때문이다(§ 4.10).

(7) branching policy의 비용 기반 fine-tuning. branching 결정의 정확한 비용 — 그 subtree가 전개하는 node 수 — 은 탐색 자체에서 얻어지며, held-out 상태에서 확신도 규칙이 후보 5개 중 가장 싼 것을 고르는 경우는 22%뿐이다. 그 비용에 대한 actor-critic fine-tuning은 지도 규칙에서 출발해 marginal network를 동결한 채, 100개 세트에서 지도 policy보다 node를 2.3배 적게 전개하고 2.0~2.2배 빠르며(70/100, 64~67/100 우세, 모든 seed에서 $p < 0.01$), LP 기반 branching보다 4.8~5.9배 빠르다(§ 4.11).

2. 선행연구

2.1 Binary linear system과 market-split의 난이도

식 (1)은 0/1 integer feasibility 문제이며 일반 A에 대해 NP-complete이다. 그러나 실무적 난이도는 instance 분포에 달려 있다. Cornuéjols와 Dawande(1999)는 market-split 계열을 도입했다: m 명의 agent가 n 개 품목을 각자의 예산에 정확히 맞게 나눠야 하며, $n \approx 10(m-1)$ 일 때 놀랍도록 작은 크기에서 branch-and-bound를 무력화한다. 메커니즘은 integrality gap이다: relaxation polytope이 크고 그 vertex가 fractional이라 bounding이 거의 가지치기를 못 하고, 탐색은 사실상 눈먼 채로 분기해야 한다. Aardal 등(2000)은 lattice basis reduction이 같은 계열을 branch-and-bound보다 훨씬 효과적으로 공략함을 보였고, 우리는 § 3.2에서 이를 활용한다.

우리 생성기(§ 4.1)는 relaxation vertex들의 퍼짐(spread)을 최대화해 이 메커니즘을 명시적으로 만든다. 이는 S_{LP} 를 키우고 relaxation signal을 설계 단계에서 약화시킨다.

2.2 Complete solver

Mixed-integer programming, linear relaxation 위의 branch-and-bound를 presolve와 cutting plane으로 강화하는 것이 표준이다. 그 지렛대는 relaxation이 infeasible하거나 target에서 멀리 떨어진 node를 가지치는 데서 나오는데, market-split 구성이 제거하는 것이 정확히 그것이다.

Clause learning을 갖춘 constraint programming. CP-SAT 계열 solver는 constraint propagation에 conflict-driven clause learning(CDCL)과 lazy clause generation을 결합한다. 이 계열에서의 강점은 본 논문의 논지에 핵심적인 성질에서 나온다: 탐색은 내려가면서 정보를 만들어낸다. $\Pr[x_j=1 \mid A, b]$ 가 root에서 무정보해도 $\Pr[x_j=1 \mid A, b, x_1=1, x_5=0]$ 은 0이나 1일 수 있다. root에서 보이지 않던 구조가 commitment 이후 연역 가능해지며, 이것이 § 3.4가 root가 아니라 tree 내부를 다루는 이유다.

Lattice reduction. 대안적 접근은 식 (1)을 lattice에 embedding하고 짧은 vector를 찾는다. LLL은 다항시간에 basis를 축소하고, BKZ는 block size로 시간과 품질을 교환하며 이를 일반화한다. Aardal-Hurkens-Lenstra(AHL) embedding은 0/1 해가 짧은 vector로 직접 나타나도록 구성되며, reduction이 heuristic이 아니라 certificate를 생산하는 절차가 된다. lattice enumeration은 정식 벤치마크에서 여전히 최고 수준이다: Wassermann(2025)은 QOBLIB market-split instance를 CPU 하나로 $m=14$ 까지 푸는데 branch-and-cut은 $m=7$ 에서 멈춘다; 우리 instance는 다른 영역(multiplicity 일치, $m/n \approx 0.35$)이고 우리 기여는 그 기록이 아니라 branching 방법론이다. 우리는 lattice reduction을 전체 pipeline의 첫 단계로 사용한다.

2.3 조합최적화를 위한 학습

Bengio 등(2021)은 이 분야를 개관하며 우리가 채택하는 표현을 쓴다: solver는 "계산하기에 너무 비싸거나 수학적으로 잘 정의되지 않은 결정"에 대해 수작업 heuristic에 의존하며, 학습이 그 결정을 개선할 후보라는 것이다. too expensive to compute라는 표현은 그 결정이 원리적으로 계산 가능함을 전제하고 비용만을 묻는다 — 이것이 § 4.5에서 우리가 측정하는 상황이다.

해집합의 marginal을 branching 신호로. 결정에 동의하는 해의 비율은 학습 이전부터 분기 기준이었다: counting-based search(Pesant et al., 2012)는 개별 제약 안에서 계산한 solution density가 최대인 변수-값으로 분기하고, belief-propagation-guided decimation(Montanari et al., 2007)은 고정할 때마다 다시 계산한 근사 marginal로 가장 편향된 변수를 고정한다. 학습 버전은 counting을 network로 대체한다: NSNet(Li and Si, 2022)은 SAT 공식의 만족 배정을 열거해 얻은 정확한 marginal로 BP 구조의 graph network를 학습해 local search 초기값으로 반올림하고, MIP-GNN(Khalil et al., 2022)·Neural Diving(Nair et al., 2020)·predict-and-search(Han et al., 2023)는 수집한 준최적 해의 변수별 bias를 예측해 변수 고정·warm start·탐색 제한에 쓴다. 우리 target은 NSNet과 같다; 다른 것은 marginal이 conditional이고 reduction 항등식으로 모든 node에 옮겨지며 아무것도 확정하지 않고 complete search를 안내한다는 점, 그리고 FORCED/OPEN 분할과 in-tree 대 root 비교에 대응물이 없다는 점이다.

Branching imitation. 이 분야의 가장 분명한 성공은 비싼 expert의 모방이다. strong branching은 두 자식의 relaxation을 잠정적으로 풀어 후보에 점수를 매기며, 작은 tree를 만들지만 매 node에서 돌리기에 너무 느리다. Gasse 등(2019)은 bipartite graph network에 이를 모방시켜 훨씬 적은 비용으로 이득 대부분을 회수했다. 이득은 새 정보가 아니라 — expert는 이미 답을 알고 있었다 — 그 지식에 값싸게 접근할 수 있게 된 것이다. 우리 방법은 같은 구조적 위치를 차지하되 두 가지가 다르다: target이 expert의 점수가 아니라 정확한 posterior이며, 분석이 expert의 결정 중 어느 것이 중요한지를 식별한다.

직접적 satisfiability 예측. network가 satisfiability를 종단으로 판정하게 하려는 시도는 기록이 더 약하다. NeuroSAT은 literal-clause graph 위의 message passing으로 분류하며 작은 random 인스턴스에서 성공하지만, G4SATBench는 크기 transfer에서 상당한 정확도 붕괴를 보고한다. Chen 등(2023)은 표준 message-passing network가 어떤 feasible/infeasible MILP 쌍을 원리적으로 구분할 수 없음을 증명했다. 따라서 우리는 network에게 feasibility 판정을 요구하지 않는다. network는 건전성을 책임지는 complete search 내부에서 branching guidance만 제공한다.

branching을 위한 강화학습. 모방은 expert의 순위를 target으로 삼지, 그 순위가 줄이려는 탐색 비용을 target으로 삼지 않는다. FMSTS(Etheve et al., 2020)는 node의 subtree 크기를 관측 가능한 Q-value로 쓰고 depth-first search에서는 모든 subtree의 최소화가 전체 tree의 최소화임을 보이며 from scratch로 학습한다; tree MDP(Scavuzzo et al., 2022)는 그 subtree 단위 credit assignment를 형식화하고; retro branching(Parsonson et al., 2023)은 회고적으로 추출한 subtree 경로에서 학습하며; SORREL(Feng and Yang, 2025)은 준최적 시연으로 초기화하고; Neuro#(Vaezipoor et al., 2021)은 #SAT solver의 residual formula 위 graph network를 evolution strategies로 학습해 branching 횟수를 최소화한다. constraint programming에서는 value ordering을 해로부터(Chu and Stuckey, 2015) 또는 deep Q-learning으로(Cappart et al., 2021) 학습했고, impact-based search(Refalo, 2004)는 배정이 탐색공간을 가장 많이 줄이는 변수를 선호한다. 우리의 fine-tuning 단계 (§ 3.8)는 같은 subtree 단위 credit을 단순 policy-gradient(Williams, 1992)로 쓰되 출발점이 다르다: policy는 정확한 conditional marginal에서 유도된 지도 규칙으로 초기화되고 marginal network는 동결되므로, 비용 신호가 바꿀 수 있는 것은 어느 변수로 분기하는가뿐이다. 이 목적함수의 고전적 진술이 fail-first 원칙(Haralick and Elliott, 1980)이다: 틀린 branch가 가장 빨리 반증되는 변수를 골라라.

2.4 Amortized optimization

우리 결과를 조직하는 구분은 조합최적화에서 쓰이기 전부터 있었다. Gershman과 Goodman(2014)은 amortized inference를 도입했다: 관련된 query가 많을 때 매번 처음부터 푸는 것은 낭비이며, 질문은 query 간에 계산을 어떻게 재사용하는가가 된다. Amos(2023)는 같은 아이디어를 amortized optimization으로 전개한다 — 유사한 problem instance 간의 공유 구조를 활용해 학습으로 해를 예측하는 것 — 그리고 variational inference, meta-learning, control에서의 등장을 개관한다.

Millidge(2022)는 그 trade-off를 여기서 가장 관련 있는 형태로 진술한다: direct optimization은 당면 instance에 계산을 쓰며 호출할 때마다 그 비용을 지불하고, amortized optimization은 문제를 supervised learning으로 바꿔 inference를 값싸게 만들지만 풀린 인스턴스 corpus가 선결 조건이고 approximator의 generalization에 갇힌다. 생산적인 배치는 hybrid다 — amortized 예측이 direct optimizer를 안내한다.

우리 시스템이 그 hybrid를 정확히 구현한다. direct 방법은 LP-probing이며 건전하되 비싸다. amortized 방법은 network이며 값싸되 건전하지 않다. hybrid 규칙은 network가 branching을 안내하되 probing과 propagation이 변수 확정에 대한 배타적 권한을 유지하는 것이다 (§ 3.7).

3. 제안 방법

3.1 설정과 표기

변수 집합 F 를 값 v 로 고정하는 partial assignment에 대해, 그와 부합하는 해집합을 $S(F, v) = \{x \in S : x_F = v\}$ 라 쓰고, 자유 변수 집합을 K 라 한다. 자유 변수 $j \in K$ 의 conditional marginal은 다음과 같다:

$$p_j(F,v) = |\{x \in S(F,v) : x_j=1\}| / |S(F,v)| \quad (2)$$

3.2 시스템 개요

실제 solver는 lattice 단계와 guided complete search의 cascade다:

propagation → LP infeasibility rule → kernel pump → AHL/BKZ → guided backtracking search

앞의 세 단계는 값싸고 건전하다. AHL/BKZ는 크기 의존적 block 파라미터로 certificate를 생산하는 lattice reduction을 시도하며, 성공하면 검증된 해를 바로 반환한다. 학습 컴포넌트는 마지막 단계에만, 그것도 branching 결정에만 나타난다. 이 배치는 평가에서 중요하다: lattice 단계가 branching heuristic과 무관하므로 비교 대상인 모든 heuristic이 정확히 같은 잔여 instance 집합을 받으며, 따라서 그 잔여 위에서 조건부로 보는 것은 유리한 사례를 고르는 것이 아니라 알고리즘을 기술하는 것이다.

3.3 Conditioning은 reduction이다

명제 1. A_F 와 A_K 를 각각 F, K 로 인덱싱된 열 부분행렬이라 하자. 그러면

$$S(F,v) \cong \{y \in \{0,1\}^{|K|} : A_K y = b - A_F v\} \quad (3)$$

이며 대응은 $x \rightarrow x_K$ 이다. 따라서 원본 instance의 conditional marginal (2)는 축소된 instance의 통상적 marginal이다.

증명. $Ax = A_F x_F + A_K x_K$ 이다. $x_F=v$ 로 고정하면 $Ax=b \Leftrightarrow A_K x_K = b - A_F v$ 이고, x_F 가 결정되어 있으므로 $x \rightarrow x_K$ 는 두 집합 사이의 전단사다. marginal에 대한 진술은 개수를 세면 따라 나온다. □

이 항등식은 초등적이지만, in-tree 지도학습을 실행 가능하게 만드는 것이 바로 이것이다. depth-d search node는 자기만의 encoding을 요구하는 특수한 대상이 아니라, 같은 문제의 n-d개 변수 instance일 뿐이다. 따라서 하나의 network, 하나의 feature 추출기, 하나의 label 생성기가 tree 전체를 담당하며, depth d의 학습 데이터는 solver를 계측하는 것이 아니라 대입으로 생산된다.

두 구현 세부가 정합성을 유지한다. 대입 후 전부 0이 된 row는 제약을 부과하지 않으므로 제거한다 — 남겨두면 linear programming backend가 모델을 거부한다. 그리고 자유 bound 조건 $0 \leq b - A_F v \leq A_K 1$ 을 relaxation을 풀기 전에 검사한다. 무시할 만한 비용으로 dead node의 상당 부분을 걸러내기 때문이다.

3.4 어떤 예측이 비용을 발생시킬 수 있는가

살아남은 해집합이 비어 있지 않은 node에서, 자유 변수를 conditional marginal로 분할한다:

$$DET = \{j : p_j \in \{0,1\}\}, \quad OPEN = \{j : 0 < p_j < 1\}$$

이 구분은 통계적이 아니라 운용적이다. $j \in OPEN$ 이면 x_j 로 분기한 두 자식 모두 해를 포함하므로, 어느 쪽을 골라도 해가 도달 가능한 채로 남는다 — 즉 OPEN 변수에서의 어떤 branching 결정도 비용을 발생시킬 수 없다. $j \in DET$ 이면 한쪽 자식이 비어 있다. 강제된 값을 거슬러 분기하면 subtree가 해 없이 소진되는 것이 확정되며, 이는 온전한 backtrack 비용이다.

이것이 conditional marginal의 운용적 독법을 준다. 그것은 지정된 어떤 해에 대한 "맞힐 확률"이 아니라 그 배정으로 살아남은 해집합의 비율이며, 중요한 오류는 그 비율이 0이거나 1인 변수에서의 오류뿐이다.

두 가지 귀결이 따르고, 실험이 둘 다 확인한다. 첫째, 모든 자유 변수에 대한 총계 정확도는 나쁜 목적함수다. backtrack을 유발할 수 있는 결정과 그럴 수 없는 결정을 섞기 때문이다. 관련 있는 양은 DET에서의 정확도다. 둘째, 탐색이 내려갈수록 $|S(F,v)|$ 가 줄어들므로 DET 비중이 depth에 따라 커진다. 다만 이것만으로는 학습된 predictor가 어디서 유용한지 말할 수 없다 — relaxation이 이미 올바르게 반올림하는 DET 변수에는 predictor가 필요 없기 때문이다. 유용한 영역은 DET 변수가 많으면서 동시에 relaxation이 약한 곳이며, § 4.9는 이를 지배하는 것이 depth 자체가 아니라 제약 밀도 $m/|K|$ 임을 보인다: 작은 instance에서는 둘이 일치하지만 큰 instance에서는 그렇지 않다.

3.5 학습 target으로서의 정확한 conditional marginal

전수열거가 가능한 크기의 instance에서는 constraint solver의 all-solutions 모드로 S를 정확히 얻고, 식 (2)를 개수로 계산한다. 학습은 그 결과 얻어지는 Bernoulli target에 대한 cross-entropy를 최소화한다:

$$L = -(1/|K|) \sum_j [p_j \log p_j^* + (1-p_j) \log(1-p_j^*)] \quad (4)$$

이는 정확히 $p_j^* = p_j$ 에서 최소화된다. p_j 자리에 심어진 해를 넣는 관례적 선택은 $|S| > 1$ 일 때마다 잡음 섞인 대리 target이 된다. 심어진 해는 posterior에서 뽑은 한 표본이며, 우리 인스턴스에서 Bayes-optimal 예측조차 그것과 약 2/3만 일치한다.

열거 정합성. 여기에 놓치기 쉬운 함정이 있고, 그것을 놓치면 파생된 모든 label이 조용히 오염된다. constraint solver는 "탐색 소진"과 "해를 이미 모은 상태에서 시간제한 도달"을 서로 다른 status로 보고한다. 전자만이 S가 완전함을 보증한다. 후자를 인정하면 잘린 해집합이 대입되어 그로부터 계산된 모든 marginal이 편향된다. 우리 구현은 OPTIMAL과 INFEASIBLE만 인정하고 시간 초과한 instance는 평균에 섞지 않고 폐기한다. 유용한 진단 지표는 평균 열거 시간이 시간제한과 같아지는 것인데, 이는 잘린 instance가 완료로 집계되고 있다는 신호다.

도달 가능한 상태의 표집. depth-d 학습 상태는 d를 {0, ..., 12}에서 균등하게 뽑고, LP 확신도 순서의 앞 d개를 취해, 무작위로 고른 실제 해의 값으로 고정해 만든다. 임의의 값으로 고정하면 $S(F, v) = \emptyset$ 인 상태가 생기는데, 그런 상태는 propagation이 즉시 가지치는 dead node이고 conditional marginal이 정의되지 않으며, branching heuristic이 질문받을 일이 없는 상태다. 이 depth 범위는 튜닝하지 않은 설계 선택이며, 10×25 instance에서 자유 변수 13~25개의 학습 상태를 만든다. 배포된 탐색이 그 범위 밖의 상태를 질의하면 — 21×60에서는 대부분이 그렇다 — 어떻게 되는지는 § 4.9가 다룬다.

3.6 Network

instance는 n개 variable node, m개 constraint node, 그리고 $a_{ij}=1$ 인 곳마다 edge를 갖는 bipartite graph로 encoding된다. 현재(축소된) instance의 relaxation 해를 x^* , row sum을 r_i 라 하면 node feature는 다음과 같다:

$$\begin{aligned} \text{변수 } j: & (x_j^*, \deg(j)/m, |x_j^* - \text{round}(x_j^*)|) \\ \text{제약 } i: & (b_i/r_i, r_i/n, (b_i - A_i x^*)/r_i) \end{aligned}$$

즉 변수에는 relaxation 값·열 차수·fractionality, 제약에는 tightness·scaled row size·relaxation 잔차다. 모두 크기 정규화되어 있으며, 이것이 하나의 파라미터 집합이 임의의 (m,n)에 적용될 수 있게 한다.

두 node 유형 모두 선형 사상으로 h=64차원에 embedding되고, graph attention을 쓰는 양방향 message passing L=4라운드로 정련된다. 각 라운드는 변수→제약, 제약→변수 순이며 각각 LayerNorm과 residual 연결을 갖는다. 이어 2층 head가 변수당 logit 하나를 내어 $p_j^* = \sigma(z_j)$ 를 준다. 모델 파라미터는 21,761개다.

architecture는 의도적으로 작고 단일 head다. 유일한 출력은 목적함수 (4)가 정의하는 양이며, 보조적인 value나 feasibility head가 없다. 탐색은 건전성을 자신의 symbolic 구성요소에서 얻고 network에게는 그 외의 어떤 것도 요구하지 않기 때문이다.

3.7 추론: commitment가 아니라 guidance

network는 node당 한 번 호출된다. branching은 다음을 선택한다:

$$j^* = \operatorname{argmax}_j |p_j^* - 1/2|, \text{ 첫 시도 값} = 1[p_j^* \geq 1/2]$$

즉 network가 가장 확신하는 변수를, 예측한 값으로 먼저 시도한다. 탐색은 complete하게 유지되고 잘못된 예측의 대가는 backtracking뿐이다.

변수 확정은 건전한 deduction만이 수행한다. 두 메커니즘이 있다: constraint propagation, 그리고 반대값 배정이 relaxation조차 infeasible하게 만들 때 x_j 를 고정하는 LP-probing이다. 이 분업은 § 4.5의 측정에서 직접 따라 나온다: 오류의 대가가 시간인 곳에서는 값싼 비건전 predictor가 낫고, 오류의 대가가 정확성인 곳에서는 비싼 건전 절차가 필수이며 대체 불가능하다.

Algorithm 1 — Guided backtracking search

```

1: instance (A,b), guidance model f, T
2: stack ← [(A,b)]
3: while stack ≠ ∅ and T < T:
4:   (A',b') ← pop(stack)
5:   if b' < 0 or b' > A'.1: continue // bound
6:   A' ← A' - 0 row; row return
7:   propagate; continue
8:   if propagation: push reduce(A',b',D); continue // 1
9:   if : return , continue
10:  p^ ← f(A',b') // forward pass,
11:  j* ← argmax |p^_j - 1/2|; v ← 1[p^_j* ≥ 1/2]
12:  push reduce(A',b',{j*=1-v}), reduce(A',b',{j*=v})
13: return unresolved

```

3.8 branching policy의 비용 기반 fine-tuning

Algorithm 1의 규칙은 network가 가장 확신하는 변수로 분기한다. 이는 첫 자식이 해를 포함할 확률을 최대화하지만, 첫 자식이 해를 포함하지 않을 때 탐색이 얼마를 지불하는지, 자식이 얼마나 작아지는지는 말하지 않는다. 둘 다 subtree의 성질이고, 탐색은 그것을 부산물로 만든다: node u에서의 branching 결정의 비용은 u 아래에서 해를 찾거나 subtree를 소진할 때까지 전개한 node 수이며, depth-first search는 u에서 backtrack해 나올 때 그 값을 정확히 돌려준다. 우리는 이 비용으로 어느 변수로 분기할지를 fine-tuning한다.

결정 과정. 상태는 propagation 후의 축소 instance (A',b'), 행동은 자유변수 j이며, 먼저 시도하는 값은 marginal network의 $1[p^_j \geq 1/2]$ 로 두고 바꾸지 않는다. u에서의 행동의 return은 $-\log c(u)$, c(u)는 subtree 비용이다. 모든 결정이 자기 subtree를 가지므로 credit은 episode가 아니라 결정 단위로 배정되며, 이것이 우리가 앞서 시도했던 sparse-reward 형식들이 학습 불가능했던 이유를 해소한다.

Policy와 value. 동결한 MarginalNet trunk 위에 head 둘을 엮는다. policy logit은

$$g_j(s) = \log |p^_j - 1/2| + h_\phi(h_j)$$

이며, h_ϕ 는 변수 embedding h_j 위의 2층 network로 마지막 층을 0으로 초기화한다. 따라서 $\pi(j|s) \propto \exp(g_j/\tau)$ 는 지도 규칙을 부드럽게 한 것에서 출발하고 그 argmax는 정확히 Algorithm 1이다. value head $V_\psi(s)$ 는 pooling한 graph embedding과 $\log|K|$, $\log m$ 을 읽어 $\log c$ 를 예측한다.

갱신. 매 epoch 학습 상태를 뽑아 표집 policy($\tau=1$)를 해가 나올 때까지 전개하고 모든 결정을 정확한 비용과 함께 기록한다. epoch 안에서 표준화한 advantage $\alpha(u) = V_\psi(s_u) - \log c(u)$ 로 손실은

$$L = \sum_u [-\alpha(u) \log \pi(j_u|s_u) - \lambda H(\pi(\cdot|s_u)) + (V_\psi(s_u) - \log c(u))^2]$$

이고 entropy 가중치 $\lambda=0.01$ 이다. ϕ 와 ψ 만 학습하며 trunk와 marginal head는 고정하므로, 첫 값과 § 3.4의 분석에 쓰이는 marginal은 변하지 않는다. critic은 actor를 갱신하기 전에 지도 policy 하에서 기록한 결정으로 사전학습하고, 그 held-out 순위 상관을 관문으로 쓴다: baseline이 비용 순위를 못 매기면 advantage는 잡음이므로 fine-tuning을 시도하지 않는다. epoch 선택은 학습 분포의 held-out 상태에서의 총 subtree 비용으로 하며 평가 instance는 결코 쓰지 않는다. 배포 시 policy는 greedy로 쓴다: $j^* = \arg\max_j g_j(s)$.

3.9 비용 모델

설계를 뒷받침하는 비교는 instance 전체 판정 1회당 node 비용이며, 그 구조는 상수배가 아니라 점근적이다. node의 자유 변수 수를 n_c 라 하자. network는 feature를 만들기 위해 relaxation을 한 번 풀고 forward pass를 한 번 수행하므로 LP 호출 수가 n_c 에 대해 $O(1)$ 이다. LP-probing은 후보 변수마다 하나씩 $O(n_c)$ 회 푼다. 따라서 둘의 비율은 instance 크기에 비례해 커지며, 이 비율로 인용되는 단일 수치는 그것이 측정된 n_c 와 함께여야만 의미가 있다.

이는 또한 이 비교가 LP backend 구현에 민감하다는 뜻이며, 그 민감성은 학습 방법에 불리한 방향으로 작용한다. 호출마다 모델을 재구성하는 backend는 probing의 비용을 $O(n_c)$ 배수만큼 부풀리면서 network에는 한 번만 청구해 우위를 과장한다. 제대로 구현된 branch-and-bound는 relaxation 하나를 유지하고 bound 변경 후 부모 basis에서 재최적화하며, 이는 probe당 한 자릿수 이상 싸다. § 4.5가 양쪽 모두 warm start한 backend로 측정하고 비율을 단일 수치가 아니라 n_c 의 함수로 보고하는 이유가 이것이다.

4. 실험

4.1 인스턴스와 solution multiplicity 통제

인스턴스는 GenHard(m, n, K) 절차로 생성한다: 밀도 1/2의 무작위 0/1 행렬 A 를 뽑고, $K=20$ 개의 후보 해를 심어 각각 b 를 계산한 뒤, vertex spread(무작위 objective 방향에서 얻은 relaxation vertex들의 평균 쌍거리)가 최대인 것을 채택한다. vertex spread 최대화는 S_{LP} 를 직접 키우며, market-split 인스턴스를 relaxation 기반 방법에 어렵게 만드는 메커니즘이다 (§ 2.1). 심어진 해의 유일성은 강제하지 않는다.

왜 크기 간 $|S|$ 를 통제해야 하는가. solution multiplicity는 예측 과제의 난이도만이 아니라 성격을 바꾼다. $|S|=1$ 이면 모든 자유 변수가 DET이고 잘 예측하는 것이 곧 푸는 것이지만, $|S|$ 가 크면 대부분이 OPEN이고 과제는 퍼진 posterior를 추정하는 것이 된다. 따라서 $|S|$ 를 통제하지 않고 크기를 비교하면 크기와 과제 정체성이 교락된다. 고정된 n 에서 m 이 오르면 $|S|$ 가 줄어들므로, 크기마다 m 을 골라 이를 맞춘다.

크기	m	m/n	$ S $ 중앙값	root ceiling	conditional ceiling
10×25	10	0.40	23	70.4%	87.0%
18×50	18	0.36	19	71.3%	87.2%
21×60	21	0.35	10	—	—

표 1. 사용한 크기와 맞춰진 solution multiplicity. ceiling (§ 4.3의 Bayes-optimal per-variable 정확도)이 두 학습 크기 간에 1%p 이내로 일치하므로, 변하는 변수는 크기다.

통제의 한계. $|S|$ 를 맞추면 m/n 이 변한다(0.40→0.35). 고정된 n 에서 둘을 동시에 유지할 수 없고, 과제 정체성을 결정하는 $|S|$ 를 우선했다. 통제는 또한 입자적이다: $n=60$ 에서 $|S|$ 중앙값은 $m=20, 21, 22$ 에 대해 42→12→2로 계단을 이루므로 $m=21$ 이 사용 가능한 최선의 정수이며, 실제 평가셋은 중앙값 10에 27/30이 검증됐다.

열거가 멈추는 지점. $n=70$ 에서는 목표 multiplicity를 주는 m 에서 120초 내에 열거가 하나도 완료되지 않았다(0/10). $n=100$ 에서는 장애가 더 근본적이다: 심어진 해가 있음에도 $m \in \{28, 32, 36, 40\}$ 에서 constraint solver가 20초 내에 해를 하나도 찾지 못한다. 그 크기에서 병목은 분기를 잘 고르는 것이 아니라 해를 하나라도 찾는 것이므로 branching 비교가 유익하지 않다. 따라서 $n \leq 60$ 에서 평가하며 이 경계를 한계로 보고한다.

4.2 구현과 프로토콜

소프트웨어. 학습 feature와 원래 탐색 루프의 relaxation은 PySCIPOpt를 통한 SCIP로 풀고, § 4.5· § 4.6의 warm start relaxation은 Gurobi 13(dual simplex, bound 제자리 변경)을 쓴다 — 라이선스가 필요하며, HiGHS가 동등한 incremental 인터페이스를 제공하므로 무료 대체가 가능하겠으나 그 타이밍은 검증하지 않았다. 열거와 CP-SAT baseline은 OR-Tools, lattice reduction은 fpylll(LL, BKZ)이다. network는 PyTorch와 PyTorch Geometric(GATConv)으로 구현했다. relaxation의 한 성질을 적어둔다: objective가 없으므로 그 해는 relaxation polytope의 임의의 vertex이고 LP 코드마다 다른 vertex를 돌려준다. 한 실험 안에서 두 branching arm은 같은 vertex를 소비하므로 비교는 공정하지만, SCIP 기반과 Gurobi 기반 실행 사이의 절대 node 수는 다르다.

학습. Adam, learning rate 10^{-3} , 20 epoch, in-tree 상태 10,000개, batch size 1(instance 크기가 달라서), gradient norm clipping 1.0, 정확한 marginal target에 대한 손실 (4). 학습 상태는 depth 0~12에서 균등하게, 원본 instance당 6개씩, 1,760개 instance 풀에서 뽑는다. 평가는 분리된 풀의 440개 instance를 쓴다. 학습은 CPU 코어 1개로 약 20분 걸린다. 하이퍼파라미터는 튜닝하지 않았다: 폭·깊이·learning rate·epoch 수·학습 depth 범위는 처음 시도한 값이며 validation split이 없다. 따라서 보고 수치는 test set 선택으로 오염되지 않았으나, 이 설정이 최적이라는 주장도 하지 않는다.

스레딩. 타이밍에 민감한 모든 구성요소는 단일 스레드로 실행한다(`torch.set_num_threads(1)`). 이 크기의 graph에서 다중 스레드 BLAS는 역효과다: forward pass가 8스레드에서 11.3ms, 1스레드에서 0.9ms로 측정된다.

탐색 평가. 각 arm은 동일한 instance에 대해 subprocess 종료로 강제되는 인스턴스당 hard wall-clock 예산으로 실행된다. lattice reduction이 in-process 타이머로 중단할 수 없는 blocking native 호출 안에서 실행되기 때문이다.

instance는 20코어 머신에서 동시성 6으로 실행한다. 이 낮은 동시성은 의도적이다: wall-clock 예산 비교는 CPU 경합에 민감하며, 더 높은 동시성에서 단독 실행 시 14~15초에 풀리는 instance가 20초 상한을 넘는 것을 관측했다.

통계. 동일 instance에 대한 arm 간 해결율 차이는 McNemar 정확검정으로, 인스턴스 단위 속도 차이는 양측 부호검정으로 검정한다. 검정력이 다를 수 있으므로 단일 총계 대신 둘 다 보고한다.

평가셋. 21×60 에는 두 세트를 쓴다: 원래의 30개(생성기 seed 700000~700029)와 그것을 포함하는 100개 세트(seed 700000~700099). 후자는 30개 결과를 보고한 뒤에 만들었고 $|S|$ 는 92/100 검증, 중앙값 11이다. 100개 세트의 모든 수치는 lattice off, 3 seed다. § 4.10·§ 4.11의 학습 풀은 별도의 21×60 instance 500개(seed 800000~800499, 400/100 분할)로, 모든 평가셋과의 겹침은 (A,b) 해시 기준 0이며, 그 풀의 held-out 100개가 fine-tuning epoch 선택용 상태를 공급한다.

4.3 Bayes ceiling 대비 예측 품질

S가 정확히 열거되므로 Bayes-optimal per-variable 정확도를 계산할 수 있다:

$$\text{ceil} = (1/|K|) \sum_j \max(p_j, 1-p_j) \quad (5)$$

이는 살아남은 해들의 다수값을 답함으로써 달성되며, (A,b)로부터 x^* 를 예측하는 어떤 결정론적 predictor도 이를 넘을 수 없다. 표 2는 S가 완전히 열거된 held-out 10×25 instance 400개에서 여러 predictor를 이에 대조한다.

Predictor	전체 변수	top-3	marginal과의 L_1
Bayes ceiling	70.8%	93.2%	0.0000
제안 network	66.7%	86.6%	0.1216
LP relaxation rounding	60.9%	65.8%	—

표 2. held-out 10×25 평가 풀의 root 상태 182개에서 정확한 posterior 대비 root 수준 예측. network는 이하 모든 탐색 실험에 쓰인 것과 동일한 checkpoint다. top-3은 predictor가 가장 확신하는 변수 3개로 제한한 것이다.

network는 전체에서 ceiling의 4.1%p, top-3 부분집합에서 6.6%p 이내에 도달하며, relaxation rounding과 ceiling 사이 격차의 대부분을 회수한다(ceiling 70.8에 대해 60.9→66.7). 설계 선택과 관련된 예비 실험 둘을 완전성을 위해 보고하되, 여기서 평가한 checkpoint에서 나온 것이 아님을 밝힌다: 이전 10×25 풀에서 selection/assignment/value/feasibility 4개 head를 갖는 2.11M 파라미터 network는 root 상태에서 60.7%로 relaxation rounding(60.6%)과 통계적으로 구별되지 않았다 — 즉 용량이 구속 조건이 아니며 단일 목적 head가 낫다. 그리고 단일 head architecture를 심어진 해 대신 정확한 marginal로 학습하면 posterior와의 L_1 이 0.1261에서 0.0977로 개선되고 argmax 정확도는 변하지 않았는데, § 3.4의 분석이 argmax가 아니라 marginal을 소비하므로 이것이 중요하다.

총계가 가리는 것. 표 3은 같은 root 변수들을 § 3.4의 분할로 분해한다.

root 변수	개수	비중	부분집합 ceiling
DET (거슬러 분기하면 subtree가 빔)	38	6.3%	100.0%
OPEN (어느 쪽이든 해에 도달)	562	93.7%	67.2%
전체	600	100%	69.3%

표 3. root 수준 분해, S가 열거된 10×25 instance 24개. 총계는 그 94%가 어떤 branching 결정도 backtrack을 유발할 수 없는 변수로 이뤄져 있다.

root에서는 변수의 93.7%가 OPEN이다. 따라서 총계 ceiling 70.8%는 탐색에 영향을 줄 수 없는 예측이 지배하며, 영향을 줄 수 있는 6.3%에서는 정보가 이미 완전하다(100%). 결과적으로 root 수준 정확도는 branching heuristic에게 약한 목적함수이고, 핵심 질문은 정보가 존재하는가가 아니라 무엇이 그것을 탐지하는가이다 — 다음 절의 주제다.

4.4 Depth, 그리고 잔차가 있는 곳

명제 1을 써서, 실제 해에서 뽑은 prefix를 따라 각 depth에서 conditional ceiling과 propagation이 강제하는 남은 변수의 비율을 측정한다(표 4).

depth	살아남은 S	conditional ceiling	제안 network	LP rounding
0	25.3	70.8%	66.7%	60.9%
4	4.2	82.3%	72.8%	68.5%
7	1.8	91.5%	81.2%	77.8%
8	1.4	94.4%	84.3%	80.9%
12	1.1	98.8%	96.8%	95.5%
전체 depth	—	87.2%	80.0%	76.5%

표 4. 10×25 평가 풀의 held-out in-tree 상태에서 depth에 따른 conditional 정보량($n=25$, LP 확신도 변수 순서). 표 2와 동일한 checkpoint.

중요한 변화는 수준이 아니라 구성에 있다: DET 비중이 root의 6.3%에서 depth 8의 87.9%로 오른다(표 5). root에서는 거의 모든 branching 결정이 무해하다 — 양쪽 자식 모두 해를 포함하기 때문이다. depth 8에서는 거의 모든 결정이 틀리면 subtree를 비운다. network의 relaxation rounding 대비 우위는 전 depth에서 3~6%p이며 depth에서 사라지지 않는다 — network가 포착하는 것이 단순히 depth 의존적이지 않다는 첫 징후이며, § 4.9가 이를 정밀하게 다룬다.

다른 모든 것을 통제했을 때 in-tree 지도가 중요한가? root 학습 network와 in-tree 학습 network의 비교는 다른 것이 하나도 다르지 않을 때만 유익하다. 표 4b는 instance 풀, label 종류(정확한 marginal), architecture, optimizer, 학습 상태 수(각 1,760개)를 고정하고, 상태가 root 상태인지 depth 0~12에서 뽑은 in-tree 상태인지만 바꾼다. 둘 다 같은 held-out in-tree 상태에서 평가한다.

depth	conditional ceiling	root 학습	in-tree 학습	LP rounding
0	70.7%	67.8%	66.5%	60.9%
4	82.1%	71.4%	72.4%	68.3%
6	89.7%	74.6%	78.2%	74.0%
8	94.4%	77.4%	84.2%	81.3%
10	97.1%	81.7%	91.4%	89.4%
12	98.8%	85.8%	96.5%	95.3%
전체 depth	87.2%	75.4%	79.8%	76.4%

표 4b. 학습 상태 분포의 통제 비교. 풀·label·architecture·optimizer·상태 수가 같고 상태를 뽑는 depth만 다르다. 10×25 풀의 held-out in-tree 상태에서 평가.

in-tree 지도의 가치는 전체 4.4%p다. 더 시사적인 것은 root 학습 network 자체다: 75.4%로, in-tree 상태에서는 relaxation rounding보다 낮다. 즉 root 상태로 학습한 network는 tree 안에서 단지 뒤처지는 것이 아니라 대체하려던 classical heuristic보다 못하다. root에서 interior로의 이동은 root 학습 network가 일반화하는 분포가 아니며, in-tree 학습은 학습된 guidance를 baseline보다 낮게 만드는 요인 그 자체다.

표 5는 이 depth들에서 잔차를 분해하고 각 방법이 무엇을 탐지하는지 묻는다.

depth	%DET	network (DET)	propagation	LP integrality	LP-probing	network (OPEN)
5	61.9%	89.7%	1.2%	67.9%	51.6%	54.4%
6	74.2%	88.2%	1.6%	70.4%	68.0%	52.5%
7	79.9%	88.8%	3.7%	73.3%	72.8%	51.2%
8	87.9%	89.4%	5.3%	76.9%	79.8%	53.8%

표 5. 표 4와 같은 상태에서의 잔차 분해. 4~6열은 ground-truth DET 집합에 대한 recall(LP-probing은 § 4.5의 warm start backend로 측정), 3열과 7열은 각 부분집합에서 network의 argmax 정확도다.

잔차는 전적으로 DET 변수 위에 있다: depth 7에서 $0.80 \times 11.2 \approx 9\%$ 로, conditional ceiling 대비 측정된 10.3%p 미달분과 부합한다. OPEN 변수에서 50% 근처 성능은 결함이 아니라 올바른 거동이다 — 그 변수들은 진정으로 미결정이기 때문이다. 두드러지는 수치는 unit propagation이 논리적으로 결정된 변수의 1.2~5.3%만 탐지한다는 것이다: 정보가 존재하고 연역 가능한데도 가장 값싼 건전 규칙으로는 닿지 않는다. LP-probing은 52~80%를, LP integrality는 68~77%를 회수한다.

4.5 Node당 비용

두 방법 모두 instance 전체 판정에 도달하기까지 반드시 치러야 하는 비용 전부를 청구한다: network는 feature relaxation 한 번과 forward pass 한 번, probing은 자유 변수마다 relaxation 한 번. 양쪽 모두 동일한 warm start backend를 쓴다 — 모델 하나를 유지하며 변수 bound를 제자리에서 바꾸고 부모 basis에서 재최적화하는 방식(Gurobi dual simplex)으로, branch-and-bound 구현이 relaxation을 재사용하는 방식 그대로다. 모든 측정은 단일 스레드다.

인스턴스	n	network	LP-probing	배수	probing 탐지율
10×25	25	3.22 ms	5.60 ms	1.7×	—
18×50	50	2.33 ms	11.84 ms	5.1×	—
21×60	60	2.57 ms	18.39 ms	7.2×	—
40×100	100	4.02 ms	62.46 ms	15.5×	—
60×150	150	4.45 ms	79.06 ms	17.8×	—
10×25의 tree 내부 상태, depth별:					
depth 5	20	2.44 ms	4.02 ms	1.6×	51.6%
depth 6	19	2.39 ms	3.98 ms	1.7×	68.0%
depth 7	18	2.35 ms	3.43 ms	1.5×	72.8%
depth 8	17	2.34 ms	3.13 ms	1.3×	79.8%

표 6. instance 전체 판정 1회당 node 비용. 양쪽 모두 warm start backend, 단일 스레드. depth 5~8에서 network의 DET 변수 정확도는 85.9~92.7%이며 마지막 열의 probing 탐지율과 대조된다.

비율은 비용 모델이 예측한 대로 거동한다: $n=25$ 에서 무시할 만하고 $n=150$ 에서 17.8배에 이르며, relaxation 호출 수의 $O(n_c)$ 대 $O(1)$ 차이를 따라간다. 두 귀결이 따르는데, 서로 반대 방향을 가리킨다.

우위는 실재하지만 점근적이며, 우리가 탐색할 수 있는 구간에서는 작다. 우리 탐색이 실제로 작동하는 크기($n=25\sim60$)에서 비율은 1.3~7.2배다. 예측이 가장 중요한 depth(10×25의 5~8, $n_c=17\sim20$)에서는 1.3~1.7배다. 그 규모에서 둘 중 하나를 고르는 실무자는 probing을 택해야 한다 — probing은 건전하기 때문이다: 검사가 발화하면 결론이 증명인 반면 network는 어떤 보장도 제공하지 않는다. 학습 대안이 설득력을 갖는 것은 $n \geq 100$ 부터이고, § 4.1이 보이듯 그 구간에서는 우리가 시도한 모든 전략에서 탐색 자체가 실패한다.

측정은 baseline 구현 품질에 민감하다. 이 비교의 이전 버전은 호출마다 LP 모델을 재구성하는 backend를 썼고, 거기에 더해 network 자신의 feature relaxation을 타이머에서 제외했다. 그 조합이 depth 5~8에서 19~23배를 보고했는데 올바르게 측정한 값은 1.3~1.7배다 — 한 자릿수 이상의 과장이며, 두 방법이 계산하는 내용이 아니라 전적으로 양쪽을 어떻게 계측했는가에서 비롯됐다. 정정된 수치를 보고하며 일반적 위험을 함께 적어둔다: classical 절차와의 어떤 비용 비교에서든, classical 쪽은 실무자가 구현하듯 구현되어야 하고 학습 쪽은 자신의 전처리 비용을 청구받아야 한다.

4.6 탐색 성능

lattice 단계를 끄고 branching 품질만 격리한 채, wall-clock 예산을 맞춰 branching 전략을 비교한다. lp는 가장 정수에 가까운 relaxation 값을 갖는 변수를 그 값으로 먼저 분기하고, model은 제안 network를 쓴다. 둘은 같은 루프에서 돈다 — node당 warm start relaxation 한 번 (§ 4.2), 동일한 propagation, 동일한 값 순서 — 따라서 두 arm은 어느 변수로 분기하고 어느 값을 먼저 시도하는지만 다르다. 각 설정을 seed를 바꿔 세 번 실행하며, depth-first

search 자체는 relaxation vertex가 주어지면 결정적이므로 seed는 시간만 바꾸고 node 수는 바꾸지 않는다.

크기 (예산)	전략	해결율	node 중앙값	시간 중앙값	node/s
10×25 (60초)	lp	100±0.0%	46	0.03±0.00초	1,767
	model	100±0.0%	24	0.07±0.01초	376
18×50 (300초)	lp	100±0.0%	4,224	2.43±0.33초	2,066
	model	100±0.0%	1,085	1.82±0.37초	731
21×60 (600초)	lp	100±0.0%	82,366	51.5±2.0초	1,627
	model	100±0.0%	8,657	14.6±0.7초	619

표 7. warm start 루프에서의 branching 전략 비교, lattice 단계 비활성화, 각 칸 30개 instance, seed 3개의 seed별 중앙값에 대한 평균±표준편차. node 수는 seed에 불변이다.

세 가지를 짚는다. 첫째, 학습된 guidance는 모든 크기에서 더 작은 tree를 만들고 — lp 대비 node 2.0배, 3.9배, 9.5배 적음 — 그 비율은 크기에 따라 커진다. § 4.9의 분석이 예측하는 대로다: network가 relaxation rounding과 갈라지는 상태는 큰 tree의 위쪽에 있는 크고 약하게 제약된 상태이며, 10×25에는 그런 상태가 거의 없다. 둘째, 작은 tree가 시간을 사는 것은 tree가 forward pass 비용을 상쇄할 만큼 커진 뒤부터다. 10×25에서 model arm은 오히려 느리다(0.34~0.43배, 30개 중 최대 1개에서 우세) — 24-node tree는 그것을 줄이는 network 평가보다 싸기 때문이다. model의 node당 비용은 모든 크기에서 lp의 2.6~4.7배이며 이는 § 4.5의 forward pass다. 18×50에서 인스턴스별 시간 비율 중앙값은 network 쪽으로 1.30~1.37배이지만 유의하지 않다(모든 seed에서 30개 중 18개 우세, 부호검정 $p=0.36$). 셋째, 21×60에서 우위가 뚜렷하고 재현된다: 인스턴스별 비율 중앙값이 seed별 2.57배, 3.03배, 2.96배이고 network가 각 seed에서 정확히 30개 중 23개에서 빠르며($p=0.005$), 두 arm 모두 예산 안에 모든 instance를 푼다.

두 전략을 더, 매 node에서 relaxation을 재구성하던 이전 루프에서 측정했다. random branching은 20×50에서 300초 내 0/30을 풀었고, § 4.12은 현재 루프에서도 21×60 잔여를 하나도 닫지 못함을 보인다. lp-probe(전 자유 변수를 probing해 강제된 것을 건전하게 확정한 뒤 lp처럼 분기)는 압도적으로 작은 tree를 만들었으나(20×50에서 lp 대비 node 51배 적음) 가장 느린 arm이었다 — 초당 6 node 대 376 node. 즉 node 수와 wall-clock이 방법들을 정반대 순서로 세우며, tree 크기 관행은 최악의 방법을 고르게 된다. lp-probe를 warm start 루프에서 반복하지는 않았고, § 4.5가 그 node당 비용을 직접 측정한다.

100개 instance에서의 반복. 30개로는 부호검정의 검정력이 낮고 중앙값의 불확실성이 크므로, 21×60 비교를 100개 세트에서 반복했다(표 7b). 방향과 유의성은 유지되지만 — model이 70~71/100에서 빠르고($p<0.001$) node 5.5배 적음 — 마진은 원래 30개보다 작다: 시간 2.0~2.3배 대 2.6~3.0배, node 5.5배 대 9.5배. 원래 30개는 큰 세트의 앞 30개 seed이며 거기서 지도 policy가 유난히 잘한다. 100개 수치를 추정치로, 30개 수치를 그 표집 변동의 유리한 끝으로 본다.

전략	해결율	node 중앙값	시간 중앙값	node/s	lp 대비 (인스턴스별)
lp	100±0.0%	79,609	41.2±1.9초	1,966	—
model	100±0.0%	13,386	16.5±0.6초	796	2.0~2.3배, 70~71/100, $p<0.001$
model+ft (§ 4.11)	100±0.0%	5,495	7.6±0.3초	738	4.8~5.9배, 81~84/100, $p<0.001$

표 7b. 21×60, 100개 instance, lattice off, 600초 예산, seed 3개(seed별 중앙값의 평균±표준편차; node 수는 seed 불변). 마지막 행은 § 4.11의 비용 기반 fine-tuning policy.

재구성 루프가 21×60 비교에서 보고했던 것도 기록해 둔다. 이 논문의 이전 초고가 그것에 기대고 있었기 때문이다: 30/30 대 25/30 해결과 median 3.29배. relaxation을 warm start하자 해결율 차이는 완전히 사라졌고 — lp가 600초 안에 못 풀던 5개 instance를 이제 65~270초에 닫는다 — 속도배수도 줄었다. 해결율 우위는 node를 더 많이 전개하는 arm에 불리한 느린 backend의 산물이었고, 우리는 그것을 주장하지 않는다.

4.7 Complete solver와의 비교

표 7은 우리 탐색 루프 안에서 branching 전략을 비교한 것이라 branching 품질은 측정하지만 이 시스템이 기성 solver에 대해 어디쯤 있는지는 말해주지 않는다. 그 비교를 동일 instance·단일 스레드·동일 예산으로 여기 보고한다.

크기	CP-SAT	SCIP	제안 (model)	CP-SAT 우위
10×25	0.002초	0.009초	0.07초	35배
18×50	0.046초	0.385초	1.82초	40배
21×60	1.159초	3.687초	14.6초	13배
21×60, 100개	0.792초	2.590초	16.5초 / 7.6초 (fine-tuned)	21배 / 10배

표 8. 중앙값 해결 시간. 모든 방법이 모든 instance를 해결한다. 제안 열은 표 7의 model arm(seed별 중앙값의 평균)이고, 마지막 행은 100개 세트와 § 4.11의 fine-tuning policy를 더한 것이다. 제안 시스템은 어느 크기에서도 complete solver를 이기지 못하며, 학습 feature에 그 LP 코드를 쓰는 SCIP에게도 뒤진다.

격차가 크고, 우리는 이를 명확히 진술한다: 본 논문의 기여는 CP-SAT와 경쟁하는 solver가 아니다. 다만 격차가 어디서 오는지는 물어볼 가치가 있다. branching heuristic이 실제로 무엇을 하고 있는지에 대한 답이 거기 있기 때문이다. 표 9는 21×60에서 CP-SAT 자체의 branch 카운터를 써서 격차를 트리 크기와 node 처리량으로 분해한다.

	CP-SAT	제안	비율
node (중앙값)	18,827 branches	8,657	우리가 2.2배 적음
처리량	19,782 node/초	619 node/초	CP-SAT 32배 빠름
총 시간 (중앙값)	1.065초	14.6초	14배

표 9. 21×60 격차의 분해. CP-SAT는 추가로 instance당 중앙값 5,514개의 conflict clause를 학습한다.

학습된 guidance는 CP-SAT보다 작은 탐색 트리를 만든다 — CP-SAT의 clause learning에도 불구하고 node가 2.2배 적다 — 그리고 전적으로 node당 비용에서 진다. 그 비용이 어디로 가는지는 측정할 수 있다. warm start 이전의 루프는 초당 254 node를 처리하며 node당 4.34ms를 relaxation 재구성·풀이에 썼다. 같은 LP를 warm start하면 그것이 0.08ms가 되고 처리량은 초당 619 node로 오르는데, 이것이 § 4.6 전체에서 보고한 루프다. node당 남은 1.6ms 중 network forward pass가 root에서 1.88ms(축소 graph가 작아지는 깊은 node에서는 그보다 적음), propagation과 bookkeeping이 0.1ms 미만이다: 이제 forward pass가 node당 비용의 90% 이상이며, 우리는 그것을 줄이지 않았다. 따라서 격차가 구현 산물이라고 주장하지 않는다. node 수가 확립하는 것은 더 좁다: 부족분이 branching 품질에 있지는 않다는 것이다. 격차를 닫으려면 최소한 compiled inference 경로와, 다음 단락이 적듯 우리 탐색에 없는 알고리즘 구성요소가 필요하다.

이 해석에는 정직한 단서 두 가지가 붙는다. 트리 크기가 완전히 통약 가능하지는 않다 — CP-SAT의 "branch"에는 restart와 conflict 기반 점프가 포함되고 각 node가 우리보다 많은 propagation을 수행한다 — 따라서 2.2배는 자릿수 수준의 진술로 읽어야 한다. 그리고 경쟁력 있는 구현에는 빠른 forward pass 이상이 필요하다: clause learning과 restart가 우리 탐색에 없고, 뒤의 둘은 CP-SAT가 우리가 멈추는 크기를 넘어 확장하게 해주는 요소다.

4.8 크기 transfer

feature가 크기 정규화되어 있고 파라미터가 (m,n)에 의존하지 않으므로 하나의 network가 임의 크기에 적용된다. 표 1처럼 |S|를 맞춘 상태에서 그것이 실제로 전이되는지 검증한다.

학습 크기	평가 크기	해결율	시간 중앙값	node 중앙값
10×25	10×25	100%	0.07±0.01초	24
10×25	18×50	100%	1.82±0.37초	1,085
18×50	18×50	100%	1.48±0.15초	942
10×25	21×60	100%	14.6±0.7초	8,657
18×50	21×60	100%	23.7±0.7초	14,742

표 10. warm start 루프에서의 transfer, lattice 단계 비활성화, 행마다 30개 instance, seed 3개. 2~3행과 4~5행은 각각 평가 크기를 고정하고 학습 크기만 바꾼 것이다.

21×60 평가 크기에서, 2.4배 작은 문제인 10×25에서만 학습한 network는 18×50에서 학습한 network보다 못하지 않다. 굳이 말하면 낫다: 30개 중 18개에서 node가 더 적고 각 seed에서 18~19개에서 더 빨리 끝나며, 인스턴스별 시간 비율 중앙값은 그쪽으로 1.24~1.31배다. 어느 차이도 유의하지 않고(부호검정 $p \geq 0.20$) 둘 다 30/30을 푼다. 18×50에서도 같은 그림이다: 10×25 network가 30개 중 16~17개에서 더 빠르지만($p \geq 0.59$) 시간 중앙값은 약간 높는데, 짝지는 차이가 instance 간 변동에 비해 작아 두 요약이 엇갈리는 것이다. 재구성 루프에서의 이전 비교는 15/30과 0.92배를 주었다. 방향은 루프에 따라 안정적이지 않으며, 우리는 이 결과를 transfer 이득이 아니라 검출 가능한 transfer 손실 없음으로 읽는다. root 수준 예측도 같은 결론을 준다: 10×25 network가 10×25, 20×50, 40×100에서 각각 67.8%, 69.5%, 69.4%를 기록하며 relaxation rounding 대비 우위(+11.0, +7.2, +4.7%p)를 모든 크기에서 유지한다.

21×60 너머에서의 두 단계 전이. 표 10b는 모든 학습 컴포넌트의 학습 범위 위의 크기를 더한 것으로, 열거로 multiplicity를 검증할 수 없어 21×60의 제약 밀도($m/n \approx 0.35$)로 m 을 골랐다. Stage 1은 24×70 (3.8배)· 28×80 (2.4배)에서 LP 규칙보다 node가 적지만 시간 이득은 24×70 에서 사라지는데(둘 다 풀 instance에서 0.7~1.1배), 260,000-node tree는 그만큼의 forward pass를 지불하기 때문이다. 21×60과 18×50에서만 학습한 비용 fine-tuning policy(§ 4.11)는 24×70 에서 모든 seed에 30/30을 Stage 1보다 2.3~3.3배·2.1~2.7배 빠르게 풀고, 28×80 에서는 1,200초 안에 18개·17개를 푸는데 Stage 1은 8개, LP 규칙은 4개이며 SCIP은 중앙 112초에 30개 전부를 푼다; 그 크기가 유용한 창의 위쪽 경계다.

크기	지표	CP-SAT	SCIP	lp	model (25)	model+ft (60)	model+ft (50)
24×70 (1200초, seed 3)	해결 / 시간 중앙	30/30 / 16초	30/30 / 12초	23~25/30 / 537~644초	29/30 / 358~457초	30/30 / 125~185초	30/30 / 135~165초
	node 중앙	—	—	980,743	260,114	90,054	95,818
28×80 (1200초, seed 1)	해결 / 시간 중앙	23/30 / 270초	30/30 / 112초	4/30 / >1200초	8/30 / >1200초	18/30 / 720초	17/30 / 852초
	node 중앙	—	—	1,929,041	803,535	470,606	560,077

표 10b. 모든 학습 범위 위의 크기, lattice off, 각 30개; multiplicity 미검증, 제약 밀도 일치. 괄호는 학습 크기.

이것이 실무적으로 중요한 이유는 label 생성이 구속 조건이기 때문이다. 정확한 marginal은 열거를 요구하는데 이는 $n \approx 60$ 을 넘으면 실패한다. transfer는 label을 얻을 수 있는 곳에서 학습한 network를 그릴 수 없는 곳에 배치할 수 있음을 뜻한다.

4.9 전이된 network는 어디서 도움이 되는가

§ 4.4는 유용한 예측 영역을 10×25 instance의 depth 6~8에 위치시켰다. 그 위치가 transfer 후에도 유지되는지는 별개의 질문이며, 유지되지 않는다: 21×60에서 탐색은 자유 변수 5~60개인 상태에서 network를 질의하는데, 그중 학습 범위 13~25 안에 드는 것은 19.7%뿐이고 80.2%는 어떤 학습 상태보다 크다. 표 11은 21×60 in-tree 상태 중 15초 안에 열거 가능한 부분집합에서, 자유 변수 수별로 network와 relaxation rounding을 정확한 conditional marginal에 대조한다.

자유 변수	학습 범위 내	$m/ K $	network (DET)	LP (DET)	이득	corr(network, LP)
20	예	1.05	100.0%	100.0%	+0.0	1.000
25	예	0.84	100.0%	100.0%	+0.0	0.998
30	아니오	0.70	95.7%	95.0%	+0.7	0.954
40	아니오	0.53	75.5%	75.2%	+0.4	0.747
50	아니오	0.42	70.2%	65.0%	+5.3	0.664
60	아니오	0.35	93.3%	88.9%	+4.4	0.658

표 11. 자유 변수 수별 21×60 in-tree 상태, 행당 열거 가능 상태 9~14개. 이득은 DET 변수에서 network의 relaxation rounding 대비 마진, 마지막 열은 두 predictor 출력의 Pearson 상관이다.

패턴은 depth 기반 설명이 예측하는 것과 정반대다. 학습 범위 안에서 network는 relaxation rounding과 구별되지 않는다 — 둘 다 완벽하고 출력 상관이 1.000이다 — $m/|K| \geq 0.8$ 에서는 축소 instance가 과제약이어서 relaxation이

이미 모든 DET 변수에서 정수이기 때문이다. network의 마진 전부는 학습 중 본 적 없는 상태, 즉 자유 변수 50~60개에서 생기며, 거기서 relaxation이 가장 약하고($m/|K| \approx 0.35 \sim 0.42$) network 출력은 relaxation과 탈상관되어 있다. § 3.4의 용어로는, DET 변수가 존재하지만 relaxation이 그것을 찾지 못하는 영역이다.

두 결론이 따른다. 첫째는 § 4.4를 교정한다: depth는 10×25 에서 우연히 성립한 제약 밀도의 대리변수였다 — 거기서는 root에서 내려가도 $m/|K|$ 가 0.40에서 0.59로만 움직이지만, 21×60 에서는 같은 하강이 0.35에서 1.0을 넘어선다. 유용한 영역은 $m/|K|$ 로 정의되며, 배포 크기에서는 tree의 위쪽에 놓인다. 둘째, § 4.8의 transfer 결과는 multiplicity를 맞춘 비교만이 보이는 것보다 강하다: network는 크기와 제약 밀도 양쪽에서 학습 범위 밖의 상태로 외삽하며, 그 외삽이 기여가 있는 곳이다. 동시에 한계이기도 하다 — 학습 범위는 이를 염두에 두고 고른 것이 아니기 때문이다 (§ 4.14).

4.10 배포 크기의 상태로 학습하기: 부정 결과

§ 4.9는 network의 마진이 자유변수 50~60개 상태, 즉 학습 상태 범위 13~25 밖에 있음을 찾았고, 이전 초고의 한계 절은 둘을 맞추는 것을 명백하지만 미검증인 개선으로 적었다. 학습 상태 분포 외의 모든 것을 고정하고 이를 검증했다.

상태. 별도의 21×60 풀 (§ 4.2)에서 LP 확신도 순서의 prefix를 심어진 해로 고정해 자유변수 30, 35, ..., 60개인 in-tree 상태를 만들고 § 3.5의 열거 규칙으로 정확한 conditional marginal을 붙였다. 자유변수 55개 이하 상태는 전부 90초 안에 열거됐고, 60(root)에서는 291/400이 됐으며 실패분은 $|S|$ 가 큰 instance라 root 라벨은 작은 해집합 쪽으로 편향된다. 뒤에서 중요한 성질 하나: prefix를 해로 고정하면 자유변수가 50개 이하로 남을 때 살아남는 해집합은 거의 항상 단일원소이며($|S(F,v)|$ 중앙값 1, DET 비중 97%) 라벨이 사실상 hard다. § 4.2의 architecture·optimizer·epoch·손실로 세 network를 학습했다: M0(위의 모든 곳에 쓴 network, 10×25 in-tree 상태), M1(배포 범위 상태 2,691개만), M2(반반). 10×25 안에서만 depth 범위 0~6, 6~12, 0~12로 같은 개수를 학습한 사전진단은 in-tree 정확도 79.3%, 79.9%, 79.9%였다: 한 크기 안에서는 depth 범위가 중요하지 않다.

예측. 표 11의 분석을 세 network를 동일 상태에서 채점해 다시 하면, 자유변수 50과 60에서 DET 변수의 relaxation rounding 대비 이득은 M0 +5.4·+9.7%p, M1 +6.1·+10.7%p, M2 +5.9·+10.6%p다. 배포 영역 학습이 거기서 사는 것은 약 1%p다.

탐색. 다른 곳에서 훨씬 많이 읽는다. 100개 세트에서 M1은 node 중앙 21,337개를 전개해 M0의 13,386개보다 1.6배 많고(더 적은 경우 38/100뿐, $p=0.021$) 1.7~1.8배 느리다. 원래 30개에서는 20,746 대 8,657이다. M2는 그 사이다(18,558, $p=0.089$). 자유변수 ≥ 50 인 node를 M1에, 나머지를 M0에 보내는 switch도 아무것도 회복하지 못한다(17,252 node, 43/99, $p=0.23$). 기제는 예측 진단에 보인다: 작고 촘촘한 interior 상태(10×25 depth 6~8, 탐색이 node의 2/3를 쓰는 종류)에서 M1의 최고 확신 변수가 DET이면서 올바르게 예측될 확률은 94.4%에서 77.6%로, 확신도와 실제 마진의 순위 상관은 0.36에서 0.12로 떨어지는 반면, 배포 범위 상태에서는 개선된다(77.2→86.0%). node는 interior에 있고, tree 상단에서 조금 낮고 내부에서 훨씬 못한 network는 탐색을 느리게 한다. 학습 상태의 크기 분포는 지렛대가 아니며, § 4.9가 찾은 잔차는 분포 이동의 산물이 아니라고 결론짓는다.

4.11 비용 기반 fine-tuning

DET 변수 정확도도 학습 분포도 지도 guidance가 남겨두는 것을 설명하지 못한다면, 자연스러운 용의자는 목적함수다: 규칙은 가장 확신하는 변수로 분기하는데 확신하는 것과 싼 것은 같지 않다. 세 측정이 이를 검증하고 세 번째가 그것을 방법으로 만든다.

R0: 확신하는 선택이 가장 싼가? 자유변수 45~50개의 held-out 상태 100개에서 root 결정을 network가 가장 확신하는 변수 5개 각각으로 강제하고 그 아래를 지도 탐색으로 돌려 각 subtree의 정확한 비용을 기록했다. 확신도 규칙의 선택이 5개 중 가장 싼 경우는 22%, 가장 싼 후보 대비 비용 비율의 중앙값 1.29, 평균 2.97, 24%의 상태에서 2배 이상 싼 후보가 있으며, root에서만 가장 싼 후보를 고르더라도 이 상태들의 총 node가 55% 줄어든다. LP 규칙의 root 선택이 network의 것과 같은 비용인 경우는 10%뿐이라 두 규칙이 단순히 동률인 것도 아니다. 큰 잔차가 있고, 그것은 어느 변수로 분기하는가에 관한 것이다.

R1: 비용을 예측할 수 있는가? critic은 policy 갱신의 baseline이므로 본 적 없는 상태에서 결정을 비용순으로 매길 수 있어야 한다. 학습 풀에서 지도 탐색의 모든 결정을 기록하면 결정 37,275개(held-out 9,375개)와 정확한 subtree 비용이 나오는데 그 2/3가 자유변수 25~34개에서 일어난다. 동결한 trunk 위의 value head는 log c와 held-out Spearman 상관 0.80에 이르고, 상태 크기만 아는 baseline(log|K| 선형 적합)은 0.70이며 크기 구간 안에서는 critic의 우위가 더 크다(자유변수 35~44에서 0.68 대 0.38, 45~55에서 0.57 대 0.37). trunk를 풀면 0.02가 더해진다. § 3.8의 관문은 동결 critic으로 통과된다.

R2: fine-tuning. 자유변수 45~60개 학습 상태에서 rollout 400개씩 6 epoch($\tau=1$, 90초 예산, epoch당 약 21,000 결정). held-out monitor 상태 150개에서 greedy policy의 총 subtree 비용은 지도 규칙 대비 epoch 1~6 후 0.79, 0.49, 0.56, 0.46, 0.65, 0.56이었고 epoch 4를 선택했다. 같은 기간 policy entropy는 1.73에서 0.10으로 떨어져 선택 시점의 policy는 거의 결정론적이다.

평가. 표 7b는 100개 세트에서 fine-tuning policy(model+ft)를 지도 policy 옆에 보고한다: node 5,495 대 13,386(2.3배 적음, 70/100에서 적음, 모든 seed에서 $p<0.001$), 시간 중앙 7.6초 대 16.5초(2.0~2.2배, 64~67/100에서 빠름, $p\leq 0.007$), node당 비용은 불변(738 대 796 node/s). LP 기반 branching 대비로는 시간 4.8~5.9배, node 14배다. 원래 30개에서는 효과가 작다(node 8,873 대 8,657; 시간 1.2~1.5배, 18~21/30, $p=0.04\sim 0.36$): 추가된 70개가 이득을 만들며(node 2.5배 적음, 53/70), 이는 30개가 지도 규칙에 유리한 표본이었다는 것과 일관된다. 배치된 cascade(표 12)에서 fine-tuning policy는 lattice 잔여를 7.3~11.7초에 닫고 지도 policy는 12.2~17.7초에 닫는다.

fine-tuning policy의 반복과 전이. 다른 학습 seed(rollout 표집과 보정 head 초기화가 다름, 같은 monitor로 epoch 2 선택)로 한 두 번째 fine-tuning은 100개 세트에서 같은 그림을 준다 — node 4,940(지도 policy보다 2.4배 적음, 68/100에서 적음, $p<0.001$), 1.9~2.4배 빠름(66~68/100, $p\leq 0.002$) — 그리고 원래 30개에서는 더 큰 이득을 준다(node 4,040, 19/30에서 1.9배 적음; 2.3배 빠름, 20~21/30). 두 실행은 많은 node에서 서로 다른 변수를 고르면서 비슷한 비용에 도달하는데, 특정 해가 아니라 목적함수가 학습된 것이라면 기대되는 바다. policy는 21×60 상태로만 fine-tuning했지만 18×50 에서도 node 424 대 지도 policy 1,085(1.7배 적음, 20/30, $p=0.099$), 1.8~2.2배 빠름(22~24/30, $p\leq 0.016$)이다: 비용 기반 개선은 지도 개선처럼 작은 크기로 전이된다.

무엇이 바뀌었나. held-out 상태 300개에서 fine-tuning policy는 95%에서 확신도 규칙과 다른 root 변수로 분기한다. 그 선택은 대개 15번째로 확신하는 변수이고($|p^*-1/2|$ 0.26 대 규칙 선택의 0.43), DET이면서 첫 값이 맞을 확률도 낮다(57.5% 대 82.5%). policy는 가장 확신하는 변수에서 멀어졌고 탐색은 싸졌다. 이 개선은 root 결정 하나에 귀속되지 않는다: root만 fine-tuning 선택으로 강제하고 아래를 지도 policy로 돌리면 비용 비율 중앙값 0.93(불일치 상태 46개 중 22개에서 더 싼)이므로, 이득은 개별 결정이 아니라 결정 수열의 성질이다. 따라서 fine-tuning이 무엇을 최적화하는지는 말할 수 있어도 — 맞을 확률을 soft prior로 둔 subtree 비용 — 그것이 찾아낸 특징의 이름은 아직 붙이지 못한다. 값싼 대리 지표인 첫 자식에서 propagation이 촉발되는 양은 두 선택 사이에 차이가 없다.

4.12 전체 pipeline에서의 배치

표 7은 lattice reduction을 꺼서 branching을 격리한 것이다. 실제 배치에서는 lattice 단계가 먼저 돌고 search는 그 잔여만 본다. 그 단계가 branching heuristic과 무관하므로 모든 arm이 동일한 잔여 집합을 받는다.

크기	전략	lattice가 해결	잔여를 search가 해결	잔여 시간 중앙값	end-to-end
10×25	양쪽	30/30	—	—	100%
18×50	lp / model	24~28/30	11/11 / 11/11	0.5~5.0초 / 0.6~2.5초	100% / 100%
21×60	random	10~16/30	0/50	>600초	44.4±8.3%
21×60	lp	10~16/30	50/50	52.9~65.1초	100±0.0%
21×60	model	10~16/30	50/50	14.2~20.2초	100±0.0%
21×60	model+ft	10~16/30	50/50	7.3~11.7초	100±0.0%

표 12. warm start 루프에서의 cascade 결과, 모든 크기 seed 3개, 21×60 은 600초 예산. lattice coverage가 seed마다 달라지는데(21×60 에서 30개 중 16, 14, 10) lattice 단계가 무작위 열 순열을 뽑기 때문이며, 탐색에 넘어오는 잔여도 그에 따라 변한다. 한 seed 안에서 모든 arm은 동일한 잔여를 받는다.

lattice coverage는 크기에 따라 떨어진다 — 30/30, 24~28/30, 10~16/30 — 그리고 그 감소가 branching 품질이 중요해질 여지를 만든다. 10×25와 18×50에서는 잔여가 비어 있거나 어느 건전 전략으로도 수 초 안에 닫힌다. 21×60에서는 seed에 따라 10~20개 instance가 search로 넘어오고, 그 잔여는 진정으로 어렵다: random branching은 seed 합산 잔여 50개 중 하나도 닫지 못하므로 그 arm의 end-to-end 해결율은 lattice coverage 그 자체다. relaxation 기반과 학습된 branching은 둘 다 50개를 전부 닫으며, 둘의 차이는 시간이다. 잔여에서 학습된 guidance는 세 seed에서 14개 중 11개, 16개 중 14개, 20개 중 15개에서 더 빠르고(인스턴스별 비율 중앙값 3.48배, 2.99배, 2.38배; 부호검정 $p=0.057, 0.004, 0.041$), 합산 50개 중 40개, 비율 중앙값 3.00배($p=2.4 \times 10^{-5}$)이며 node를 6.9~9.5배 적게 전개한다. 잔여는 배치 시 branching heuristic이 실제로 작동하는 곳이고, 거기서의 속도 이득은 표 7의 격리 측정과 일치한다.

4.13 graph 구조가 기여하는가

MarginalNet은 node당 6개 스칼라를 소비해 bipartite graph 위로 전파한다. 특징 집합이 이만큼 작으면 graph를 무시하는 변수별 모델이 진지한 baseline이 된다 — 그것이 대등하다면 message passing은 장식이다. 표 13은 동일한 target·손실·분할로 학습하고 구조 사용 여부만 다른 모델들을 비교한다. 집계 변형은 각 변수에 인접한 row들의 제약측 특징 평균과 최대를 덧붙인 것으로, message passing 한 라운드가 전달할 정보에 해당한다.

모델	정확도	marginal과의 L_1	파라미터
Linear, 변수 특징만	78.1%	0.2530	4
MLP, 변수 특징만	77.8%	0.2522	8,641
MLP + 제약 집계	79.4%	0.2361	9,025
MarginalNet (bipartite graph)	84.1%	0.1514	21,761
Bayes ceiling	87.3%	0.0000	—

표 13. 10×25 instance의 tree 내부 상태에서의 ablation, 학습 2,000개·테스트 800개 상태.

message passing은 기여한다: 구조를 쓰지 않는 최선 모델 대비 +4.7%p, L_1 36% 감소이며, MLP가 남기는 ceiling까지의 잔차 중 약 60%를 회수한다. 둘 중 L_1 쪽이 더 관련이 깊은데, § 3.4의 분석이 소비하는 것이 argmax가 아니라 marginal이기 때문이다.

다만 같은 표가 주장의 한계도 정한다. relaxation 값만 쓰는 파라미터 4개 선형 모델이 78.1%에 도달하며 이는 ceiling에서 9.2%p 이내다. 가용 신호의 대부분은 그 단일 특징에 있고, graph는 그 위에 유용하지만 지배적이지는 않은 증분을 더한다.

4.14 한계

Label 생성이 평가만이 아니라 방법 자체를 제한한다. 정확한 marginal은 S의 열거를 요구하는데 이 계열에서는 $n \approx 60$ 을 넘으면 완료되지 않는다. transfer는 더 큰 크기에서의 배치를 허용하지만 거기서의 학습은 허용하지 않는다. solution sampling으로 p_j 를 근사하는 것이 자연스러운 확장이며 미검증이다.

유용한 크기 창이 양쪽에서 막혀 있다. 그 아래에서는 lattice reduction이 전부를 닫아 branching 품질이 무의미하고, 그 위에서는 탐색이 실패한다: 28×80에서 fine-tuning policy는 1,200초 안에 18/30, 지도 규칙은 8/30을 풀고 $n=100$ 에서는 constraint solver가 20초 내에 해를 하나도 찾지 못한다. 우리 시연은 $n=50 \sim 70$ 에 있으며, 60 위의 크기는 제약 밀도만 맞췄고 28×80은 seed 1개다.

우위는 시간과 tree 크기에 있지 해결율에 있지 않다. warm start 루프에서는 두 건전 전략 모두 우리가 평가한 모든 instance를 예산 안에 푼다. 지도 guidance가 21×60에서 사는 것은 100개 instance에서 median 2.0~2.3배의 속도와 5.5배 적은 node(원래 30개에서는 2.6~3.0배·9.5배, 유리한 표본)이고 fine-tuning policy가 추가로 2.0~2.2배·2.3배이며, 이는 모든 seed에서 인스턴스 단위로 유의하다. 이전 초고는 매 node에서 relaxation을 재구성하는 루프에서 5~0의 해결율 우위를 보 보였다. 그 차이는 빠른 backend에서 사라졌고 우리는 그것을 주장하지 않는다. 더 큰 예산이나 크기에서 해결율 우위가 존재하는지는 미검증이며, 크기당 평가 instance 30개로는 어차피 그것을 검출할 검정력이 낮다.

Multiplicity 통제는 근사적이다. 세 크기에서 $|S|$ 중앙값이 23, 19, 10이며, $n=60$ 에서는 연속한 정수 m 에 대해 $42 \rightarrow 12 \rightarrow 2$ 로 계단을 이루므로 더 정밀한 matching이 불가능하다. 평가 instance 30개 중 3개는 $|S|$ 가 미검증이며 그렇게 표시되어 있다. $|S|$ 를 맞추는 것은 m/n 을 0.40에서 0.35로 이동시키기도 한다.

비용 우위는 규모와 baseline 구현 품질에 의존한다. amortization 논변은 점근적으로는 성립하지만 우리 탐색이 작동하는 크기에서는 가치가 크지 않다: 예측이 가장 중요한 depth에서 warm start한 probing 대비 1.3~1.7배이며, 그 probing은 건전하기까지 하다. 15~18배에 이르는 것은 $n \geq 100$ 부터인데 그 구간에서는 우리가 시도한 모든 전략에서 탐색이 실패한다. 즉 논변이 가장 강한 영역을 우리는 시연하지 못한다.

탐색 구현이 경쟁력이 없다. 21×60 100개 세트에서 CP-SAT가 fine-tuning policy보다 9~10배, 지도 policy보다 20배 빠르다(원래 30개에서는 13배). 우리 branching이 node를 더 적게 만들므로 부족분은 node 처리량(32배)이며 이제 그것은 network forward pass가 지배한다. 경쟁력 있는 구현이 달성 가능함을 보인 것은 아니다: compiled inference 경로, clause learning, restart가 모두 없고, 뒤의 둘은 더 큰 크기에서 CP-SAT에게 결정적이다.

학습 분포와 배포 분포가 거의 겹치지 않으며, 맞추는 것은 도움이 되지 않았다. 학습 범위 $\{0, \dots, 12\}$ 는 자유변수 13~25개인 상태를 만드는 반면 21×60 탐색은 5~60개인 상태를 질의하며 질의의 80%가 25 위에 있다. § 4.10은 배포 영역으로 학습하면 탐색이 느려짐을 보인다. 큰 학습 상태를 다르게 구성하면(해가 아니라 탐색 자신의 경로에서 prefix를 뽑거나, 살아남는 해가 여럿인 상태) 달라질지는 모른다.

fine-tuning 결과는 하나의 학습 풀과 적은 학습 seed에 기대고 있다. policy는 하나의 21×60 풀의 상태로 fine-tuning했다. 학습 seed 둘이 21×60 에서 일치하고 이득은 18×50 으로 전이되지만, 더 큰 크기는 시험하지 않았고 두 seed 모두 같은 monitor로 epoch를 고른다. entropy는 4 epoch 안에 붕괴하고, epoch 선택은 원칙적 중단 규칙이 아니라 held-out monitor로 하며, policy가 이용하는 특징의 이름은 아직 붙이지 못했다(§ 4.11). 지도 policy처럼 크기 간 전이되는지는 미검증이다.

지도 network의 하이퍼파라미터는 튜닝되지 않았고 validation split이 없다. § 4.2 참조. 이 설정은 첫 시도로서 방어 가능하고 test set에 오염되지 않았으며, 적어도 depth 범위는 한 크기 안에서 중요하지 않음이 뒤에 확인됐다(§ 4.10). fine-tuning 단계는 held-out 상태에서 epoch를 선택하는데, 그 상태는 학습 풀에서 나오며 모든 평가 instance와 분리되어 있다.

단일 instance 계열. 모든 결과가 하나의 응용에서 나온 하나의 생성기에 관한 것이다. amortization 논변이 다른 constraint 계열로 전이되는지는 미검증이다.

5. 결론

본 논문은 binary linear system을 위한 학습 기반 branching heuristic과, 그것을 지도학습하는 정확한 절차를 제시했다. 두 요소가 방법을 떠받친다. BLS를 partial assignment로 conditioning하는 것은 그것을 축소하는 것과 동일하므로, 평범한 instance로 학습된 하나의 network가 구조 변경 없이 search tree의 모든 node에서 예측한다. 그리고 전수열거가 가능한 크기에서 정확한 conditional posterior는 학습 target을 공급하는 동시에 어떤 예측이 탐색에 영향을 줄 수 있는지를 식별한다: 살아남은 모든 해가 일치하는 변수만이 backtrack을 유발할 수 있으며, 그 비중은 root의 6.3%에서 depth 8의 87.9%로 오른다.

우리 탐색 루프 안에서 측정하면 heuristic은 설계대로 동작한다. 21×60 에서 relaxation 기반 branching 대비 100개 instance·세 seed 모두에서 median 2.0~2.3배 빠르고 node를 5.5배 적게 전개하며, 배치된 pipeline의 잔여를 3.0배 빠르게 닫는다 — 거기서 random branching은 아무것도 닫지 못한다. solution multiplicity를 통제하면 2.4배 작은 규모에서 학습한 network가 평가 크기에서 학습한 것과 구별되지 않게 동작하는데, 정확한 지도가 작은 크기에서만 가능하므로 이는 중요한 성질이다.

결과가 입증하지 못하는 것도 동등하게 명시한다. 이 시스템은 complete solver와 경쟁하지 못한다: CP-SAT가 시험한 모든 크기에서 9~40배 빠르다. 그 격차의 분해는 변명이 아니라 정보다 — 우리 branching이 node를 2.2배 적게 만들고 처리량에서 32배 지므로, 부족분은 node당 비용이 이제 network forward pass인, clause learning과 restart가 없는 Python 연구 구현에 있다 — 그러나 그것을 메울 수 있음을 보인 것은 아니다. 비용 논변도 우리가 처음 부여했던 무게를

지지 못한다. 제대로 warm start한 probing baseline에 대해 우리 instance에서 예측이 중요한 depth에서의 우위는 1.3~1.7배이고, 그 구간에서 probing은 network가 갖지 못한 건전성을 갖는다. 15~18배에 이르는 것은 $n \geq 100$ 부터이며 그 구간에서는 모든 전략에서 탐색이 실패한다. 이 비교의 이전 버전은 probe마다 LP를 재구성하는 backend를 쓰고 network 자신의 feature 풀이를 타이머에서 제외해 실제 값이 1에 가까운 규모에서 19~23배를 보고했다. 오류가 두 방법의 계산 내용이 아니라 전적으로 계측에서 비롯됐으므로 이 정정을 기록해 둔다.

추가 결과 둘이 지도 target이 무엇인지를 버린다. network의 예측 마진이 사는 배포 영역으로 학습 상태를 옮기는 것은 도움이 되지 않고 오히려 search tree를 두 배로 만든다 — 탐색은 옮겨진 network가 더 이상 잘 다루지 못하는 작은 interior 상태에 node를 쓰기 때문이다. 학습 상태 분포는 지렛대가 아니다. 탐색이 지불하는 비용을 최적화하는 것이 지렛대다. 모든 결정의 정확한 subtree 비용을 탐색에서 읽어, 지도 규칙에서 출발하고 marginal network를 동결한 채 branching 선택을 actor-critic으로 fine-tuning하면 100개 instance에서 지도 policy보다 node를 2.3배 적게 전개하고 2.0~2.2배 빠르며, 그 policy는 network가 덜 확신하는 변수로 분기한다. branch가 맞을 확률과 branch가 틀렸을 때의 비용은 다른 목적함수이고, 탐색이 최소화하는 것은 후자다.

남는 것, 그리고 우리가 주장하는 것은 방법론적이다. planted-solution instance 계열에서 정확한 conditional posterior는 계산 가능하며 branching에 대해 잘 정의된 지도 신호다 — 관례적 target인 심어진 해 하나는 그렇지 않다. search tree 내부에서 뽑은 상태가 root 상태보다 나은 학습 분포이고, reduction 항등식이 그 생성을 solver 계측이 아니라 대입의 문제로 만든다. 그렇게 얻은 heuristic은 solution multiplicity를 비슷하게 유지하면 instance 크기를 넘어 전이되고, 탐색에서 얻는 그 결정들의 정확한 비용은 그것을 개선하는 두 번째 잘 정의된 학습 신호다. 이것이 실용적으로 유용해지는지는 우리가 하지 않은 두 가지에 달려 있다: 전수열거 없이 marginal을 근사해 정확한 지도를 $n \approx 60$ 너머로 확장하는 것, 그리고 branching 개선이 시스템 수준에서 보일 만큼 탐색을 잘 구현하는 것이다.

재현성

코드는 proposed_src/ 아래에 구성요소별로 정리되어 있고, 공용 모듈은 util/에, 각 디렉토리의 ROLES.txt에 모든 파일 설명이 있다. 스크립트는 각자의 디렉토리에서 실행한다.

```
# |S| instance marginal
cd proposed_src/v15_bayes
python3 v15_gen_by_m.py --m 10 --n 25 --count 2200 --K 20 \
    --out ../../runs/v22/sols_10x25.json

# train/test (seed 0, 80/20)
cd ../v22_transfer
python3 v22_add_solutions.py --inp ../../runs/v22/sols_10x25.json \
    --out ../../runs/v22/full_10x25.json
python3 v22_split.py # -> full_10x25_{train,test}

# in-tree (depth 0-12, instance 6)
cd ../v17_conditional
python3 v17_make_conditional_data.py \
    --sols ../../runs/v22/full_10x25_train.json \
    --max_depth 12 --per_instance 6 \
    --out ../../runs/v22/cond_10x25_train.json

# (CPU 1 20)
python3 v17_train_conditional.py \
    --train ../../runs/v22/cond_10x25_train.json \
    --test ../../runs/v22/cond_10x25_test.json \
    --n_train 10000 --n_test 2000 --epochs 20 --target soft \
    --out ../../runs/model_n25

# : warm start, 3 seed,
# A/B/R/C/D/Crand x {lattice off, on}
cd ../v23_ablation
bash run_revision2.sh # -> runs/v23/, chain.log
python3 v23_aggregate.py # -> runs/v23/aggregate.json

# v24: (100, R0/R1/R2 fine-tuning
cd ../v24_deploy_range
```

```
bash run_v24.sh; bash run_v24_switch.sh
bash run_v24_r1.sh; bash run_v24_r2.sh
python3 v24_aggregate.py --tags M0 M1 M2 SW PV
```

모든 생성기는 seed가 고정되어 있다. 공개 checkpoint는 각 96KB다. instance 데이터와 가중치는 배포 대신 위 명령으로 재생성하며, 전체 평가 사이클은 v23_ablation/run_revision2.sh와 v24_deploy_range/run_v24*.sh에 스크립트화되어 있다.

참고문헌

- Aardal, K., Hurkens, C. A. J., and Lenstra, A. K. (2000). Solving a system of linear Diophantine equations with lower and upper bounds on the variables. *Mathematics of Operations Research*, 25(3):427–442.
- Aardal, K., Bixby, R. E., Hurkens, C. A. J., Lenstra, A. K., and Smeltink, J. W. (2000). Market split and basis reduction: Towards a solution of the Cornuéjols–Dawande instances. *INFORMS Journal on Computing*, 12(3):192–202.
- Amos, B. (2023). Tutorial on amortized optimization. *Foundations and Trends in Machine Learning*, 16(5):592–732.
- Bengio, Y., Lodi, A., and Prouvost, A. (2021). Machine learning for combinatorial optimization: A methodological tour d’horizon. *European Journal of Operational Research*, 290(2):405–421.
- Chen, Z., Liu, J., Wang, X., Lu, J., and Yin, W. (2023). On representing mixed-integer linear programs by graph neural networks. In *International Conference on Learning Representations*.
- Cornuéjols, G. and Dawande, M. (1999). A class of hard small 0-1 programs. In *Integer Programming and Combinatorial Optimization*, pages 284–293.
- Gasse, M., Chételat, D., Ferroni, N., Charlin, L., and Lodi, A. (2019). Exact combinatorial optimization with graph convolutional neural networks. In *Advances in Neural Information Processing Systems*.
- Gershman, S. J. and Goodman, N. D. (2014). Amortized inference in probabilistic reasoning. In *Proceedings of the 36th Annual Meeting of the Cognitive Science Society*, pages 517–522.
- Khalil, E. B., Morris, C., and Lodi, A. (2022). MIP-GNN: A data-driven framework for guiding combinatorial solvers. In *AAAI Conference on Artificial Intelligence*.
- Khalil, E. B., Le Bodic, P., Song, L., Nemhauser, G., and Dilkina, B. (2016). Learning to branch in mixed integer programming. In *AAAI Conference on Artificial Intelligence*, pages 724–731.
- Lenstra, A. K., Lenstra, H. W., and Lovász, L. (1982). Factoring polynomials with rational coefficients. *Mathematische Annalen*, 261(4):515–534.
- Li, Z. and Si, X. (2022). NSNet: A general neural probabilistic framework for satisfiability problems. In *Advances in Neural Information Processing Systems*, volume 35.
- Li, Z., Guo, J., and Si, X. (2023). G4SATBench: Benchmarking and advancing SAT solving with graph neural networks. *Transactions on Machine Learning Research*.
- Marques-Silva, J. P. and Sakallah, K. A. (1999). GRASP: A search algorithm for propositional satisfiability. *IEEE Transactions on Computers*, 48(5):506–521.
- Millidge, B. (2022). Deconfusing direct vs amortized optimization. *비심사 기술 노트*, <https://www.beren.io/2022-09-25-Deconfusing-direct-vs-amortized-optimization/>
- Moskewicz, M. W., Madigan, C. F., Zhao, Y., Zhang, L., and Malik, S. (2001). Chaff: Engineering an efficient SAT solver. In *Design Automation Conference*, pages 530–535.
- Montanari, A., Ricci-Tersenghi, F., and Semerjian, G. (2007). Solving constraint satisfaction problems through belief propagation-guided decimation. In *Allerton Conference on Communication, Control, and Computing*.
- Nair, V., Bartunov, S., Gimeno, F., et al. (2020). Solving mixed integer programs using neural networks. *arXiv:2012.13349*.
- Ohrimenko, O., Stuckey, P. J., and Codish, M. (2009). Propagation via lazy clause generation. *Constraints*, 14(3):357–391.
- Cappart, Q., Moisan, T., Rousseau, L.-M., Prémont-Schwarz, I., and Cire, A. A. (2021). Combining reinforcement learning and constraint programming for combinatorial optimization. In *AAAI Conference on Artificial Intelligence*.
- Chu, G. and Stuckey, P. J. (2015). Learning value heuristics for constraint programming. In *CPAIOR*.
- Feng, S. and Yang, Y. (2025). SORREL: Suboptimal-demonstration-guided reinforcement learning for learning to branch. In *AAAI Conference on Artificial Intelligence*.

- Vaezipoor, P., Lederman, G., Wu, Y., Maddison, C., Grosse, R. B., Seshia, S. A., and Bacchus, F. (2021). Learning branching heuristics for propositional model counting. In AAAI Conference on Artificial Intelligence.
- Wassermann, A. (2025). Solving the market split problem with lattice enumeration. arXiv:2508.08702; Mathematical Programming Computation, to appear.
- Schnorr, C. P. and Euchner, M. (1994). Lattice basis reduction: Improved practical algorithms and solving subset sum problems. *Mathematical Programming*, 66(1–3):181–199.
- Selsam, D., Lamm, M., Bünz, B., Liang, P., de Moura, L., and Dill, D. L. (2019). Learning a SAT solver from single-bit supervision. In International Conference on Learning Representations.
- Velickovic, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., and Bengio, Y. (2018). Graph attention networks. In International Conference on Learning Representations.
- Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4):229–256.
- Etheve, M., Alès, Z., Bissuel, C., Juan, O., and Kedad-Sidhoum, S. (2020). Reinforcement learning for variable selection in a branch and bound algorithm. In CPAIOR, pages 176–185.
- Parsonson, C. W. F., Laterre, A., and Barrett, T. D. (2023). Reinforcement learning for branch-and-bound optimisation using retrospective trajectories. In AAAI Conference on Artificial Intelligence.
- Pesant, G., Quimper, C.-G., and Zanarini, A. (2012). Counting-based search: Branching heuristics for constraint satisfaction problems. *Journal of Artificial Intelligence Research*, 43:173–210.
- Refalo, P. (2004). Impact-based search strategies for constraint programming. In *Principles and Practice of Constraint Programming (CP)*, pages 557–571.
- Scavuzzo, L., Chen, F. Y., Chételat, D., Gasse, M., Lodi, A., Yorke-Smith, N., and Aardal, K. (2022). Learning to branch with tree MDPs. In *Advances in Neural Information Processing Systems*, volume 35.
- Han, Q., Yang, L., Chen, Q., Zhou, X., Zhang, D., Wang, A., Sun, R., and Luo, X. (2023). A GNN-guided predict-and-search framework for mixed-integer linear programming. In International Conference on Learning Representations.
- Haralick, R. M. and Elliott, G. L. (1980). Increasing tree search efficiency for constraint satisfaction problems. *Artificial Intelligence*, 14(3):263–313.
- Xu, K., Hu, W., Leskovec, J., and Jegelka, S. (2019). How powerful are graph neural networks? In International Conference on Learning Representations.